

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126888

Autonómne riadenie malého vozidla s detekciou prekážky

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126888

Autonómne riadenie malého vozidla s detekciou prekážky

Bakalárska práca

Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Školiace pracovisko:	Ústav informatiky a matematiky
Školiteľ:	Ing. Štefan Bucz, PhD.



ZADANIE BAKALÁRSKEJ PRÁCE

Študent: **Rudolf Tisoň**
ID študenta: 126888
Študijný program: aplikovaná informatika
Študijný odbor: informatika
Vedúci práce: Ing. Štefan Bucz, PhD.
Vedúci pracoviska: doc. Ing. Milan Vojvoda, PhD.
Miesto vypracovania: Ústav informatiky a matematiky

Názov práce: **Autonómne riadenie malého vozidla s detekciou prekážky**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Cieľom bakalárskej práce je návrh a implementácia algoritmu autonómneho riadenia malého vozidla s detekciou prekážky pred vozidlom. Implementácia navrhnutého algoritmu je realizovaná na platforme Arduino s využitím funkcií OpenCV.

Úlohy:

1. Analyzujte súčasný stav riešenia problematiky.
2. Urobte teoretický rozbor autonómneho riadenia.
3. Navrhnite algoritmus autonómneho riadenia vozidla.
4. Realizujte navrhnutý algoritmus.
5. Overte a vyhodnoťte výsledky.

Odporúčaná literatúra:

- [1] GRIGORESCU, S.; TRASNEA, B.; COCIAS, T.; MACESANU, G. A survey of deep learning techniques for autonomous driving. Journal of Field Robotics. Issue. 37. 2019.
- [2] PAREKH, D. et al. A Review on Autonomous Vehicles: Progress, Methods and Challenges. 2022. ISSN 2079-9292.

Termín odovzdania bakalárskej práce:	05. 06. 2026
Dátum schválenia zadania bakalárskej práce:	03. 06. 2026
Zadanie bakalárskej práce schválil:	doc. Mgr. Daniela Chudá, PhD. – garantka študijného programu

Podakovanie

Na tomto mieste by som rád poďakoval svojmu vedúcemu práce Ing. Štefanovi Buczovi, PhD. za cenné rady a čas, ktorý mi venoval. Jeho odborná pomoc a podpora mali významný vplyv na výsledok mojej bakalárskej práce a boli značnou oporou pri jej písaní.

Abstrakt

Cieľom tejto bakalárskej práce bolo navrhnuť a implementovať malé vozidlo schopné autonómneho riadenia a detekcie prekážok s využitím metód počítačového videnia. Vozidlo bolo vybavené kamerovým modulom ESP32-CAM a ultrazvukovým senzorom, ktoré nepretržite monitorovali priestor pred sebou a poskytovali dáta pre riadiaci server pracujúci v programovacom jazyku Python. Dôležitým prvkom riešenia bolo spracovanie obrazu v knižnici OpenCV, kde sa prostredníctvom rozlíšenia farieb a hľadania kontúr v definovanej oblasti záujmu zabezpečovalo udržiavanie vozidla v stredovej línii vozovky. Na detekciu prekážok bola do systému integrovaná konvolučná neurónová sieť YOLOv8, ktorá spolu s meraním vzdialenosti umožnila vozidlu plynulo reagovať na objekty. Celý hardvérový systém bol riadený platformou Arduino, pričom spracovanie obrazu prebiehalo na serveri, ktorý bezdrôtovo pomocou Wi-Fi komunikoval s vozidlom.

Kľúčové slová

Arduino, OpenCV, Python, detekcia prekážky, autonómne vozidlo

Abstract

The objective of this bachelor's thesis was to design and implement a small vehicle capable of autonomous driving and obstacle detection using computer vision methods. The vehicle was equipped with an ESP32-CAM camera module and an ultrasonic sensor, which continuously monitored the area ahead and provided data for a control server operating in the Python programming language. An important element of the solution was image processing in the OpenCV library, where maintaining the vehicle within the center line of the lane was ensured through color thresholding and contour detection in a defined region of interest. For obstacle detection, a YOLOv8 convolutional neural network was integrated into the system, which, together with distance measurement, enabled the vehicle to respond smoothly to objects. The entire hardware system was controlled by the Arduino platform, while image processing took place on the server, which communicated wirelessly with the vehicle via Wi-Fi.

Keywords

Arduino, OpenCV, Python, obstacle detection, autonomous vehicle

Obsah

Úvod	11
1 Autonómne vozidlá	12
1.1 Čo je to autonómne vozidlo?	12
1.2 Úrovně autonómnych vozidiel	12
1.2.1 Úroveň 0	13
1.2.2 Úroveň 1	13
1.2.3 Úroveň 2	13
1.2.4 Úroveň 3	13
1.2.5 Úroveň 4	13
1.2.6 Úroveň 5	13
2 Ciele práce	14
2.1 Čiastkové ciele	14
3 Analýza súčasného stavu	15
3.1 Aktuálny stav technológie	15
3.1.1 Umelá inteligencia	15
3.1.2 Senzory	16
3.2 Použitie autonómnych vozidiel	17
3.2.1 Autonómne taxi služby	17
3.2.2 Autonomizácia donášky jedla	17
3.3 Zhrnutie poznatkov	18
4 Komponenty systému	20
4.1 Arduino Nano ESP32 mikrokontrolér	20
4.2 ESP32-CAM + OV2640 kamera	21
4.3 HC-SR04 Ultrazvukový senzor	22
4.4 DRV8833 driver pre motorčeky	23
4.5 Ďalšie elektrické komponenty	24
4.6 Server	24
5 Softvér	25
5.1 C++	25
5.2 Python	25
5.3 OpenCV	25
5.4 NumPy	26

5.5	CNN	26
5.5.1	Ultralytics YOLOv8	27
6	Návrh a implementácia	28
6.1	Detekcia vozovky	28
6.1.1	Region of interest	28
6.1.2	Hľadanie tmavých regiónov	29
6.1.3	Hľadanie kontúr vozovky	31
6.2	Detekcia prekážok	32
6.2.1	Ultrazvukový senzor	32
6.2.2	Ultralytics YOLOv8	33
6.3	Interpretácia dát	34
6.4	Návrh komunikácie komponentov	39
6.4.1	Komunikácia kamery a serveru	39
6.4.2	Komunikácia serveru a mikrokontroléra	41
6.4.3	Komunikácia mikrokontroléra s perifériami	41
6.5	Návrh rámu a rozloženie komponentov	43
6.6	Montovanie vozidla a nahratie softvéru	44
7	Metodika testovania	47
7.1	Autonómne riadenie	47
7.2	Detekcia prekážky	48
8	Výsledky	51
8.1	Testovanie detekcie vozovky	51
8.2	Testovanie ultrazvukového senzora	52
8.3	Testovanie konvolučnej neurónovej siete	53
	Záver	57
	Literatúra	59
	Použitie nástrojov umelej inteligencie	61
A	Výpis dlhého kódu	62
B	Výsledky testov	63

Zoznam obrázkov a tabuliek

Obrázok 1	Ako AV vnímajú prostredie	16
Obrázok 2	Arduino Nano ESP32	20
Obrázok 3	ESP32-CAM a OV2640	21
Obrázok 4	HC-SR04 ultrazvukový modul	22
Obrázok 5	DRV8833 driver pre motorčeky	23
Obrázok 6	Typická CNN	27
Obrázok 7	Obraz s použitím ROI	29
Obrázok 8	Hľadanie tmavého jazdného pruhu.	31
Obrázok 9	Diagram fungovania ultrazvukového senzora	32
Obrázok 10	Detekcia prekážky pomocou CNN.	35
Obrázok 11	Hľadanie stredu jazdného pruhu.	37
Obrázok 12	Sekvenčný diagram vyhodnotenia dát zo senzorov.	38
Obrázok 13	Diagram komunikácie komponentov	39
Obrázok 14	3D model vozidla	43
Obrázok 15	Integrované vývojové prostredie Arduino	44
Obrázok 16	Delič napätia	45
Obrázok 17	Skompletizované vozidlo (predok)	46
Obrázok 18	Skompletizované vozidlo (zadok)	46
Obrázok 19	Testovacia trať	48
Obrázok 20	Prekážky na testovanie ultrazvukového modulu.	49
Obrázok 21	Prekážky na testovanie CNN.	50
Tabuľka 1	Detekcia jazdného pruhu	51
Tabuľka 2	Ultrazvukový senzor (stred vozovky)	52
Tabuľka 3	Ultrazvukový senzor (mimo vozovky)	52
Tabuľka 4	Ultrazvukový senzor (pri vozovke)	53
Tabuľka 5	Detekcia CNN – automobil	53
Tabuľka 6	Detekcia CNN – nákladné auto	54
Tabuľka 7	Detekcia CNN – bicykel	54
Tabuľka 8	Detekcia CNN – človek	55
Tabuľka 9	Detekcia CNN – pes	55

Zoznam značiek a skratiek

ADR	Autonómny doručovací robot
AI	Umelá inteligencia
API	Application programming interface
AV	Autonómne vozidlo
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DC	Direct Current
GPIO	General-purpose input/output
GPS	Globálny polohovací systém
GPU	Graphics Processing Unit
IDE	Integrated Development Enviroment
LiDAR	Light Detection and Ranging
RADAR	Radio Detection and Ranging
RAM	Random Access Memory
ROI	Region of interest
RPM	Revolutions per minute
YOLO	You Only Look Once

Zoznam výpisov kódov

1	Funkcia ROI	28
2	Definície maxima a minima tmavých regiónov	29
3	Maskovanie tmavých regiónov	30
4	Detekcia kontúr	32
5	Obsluha prerušenia ultrazvukového senzora	33
6	Výpočet vzdialenosti z merania ultrazvukového senzora	33
7	Príprava CNN na detekciu objektov	33
8	Rozhodovanie zastavenia pri detekcii pomocou CNN	35
9	Výpočet geometrického stredu kontúry	36
10	Výpočet hodnoty riadenia	37
11	Rozhodovanie na vozidle	37
12	Zapnutie Wi-Fi hotspotu	40
13	Pingovanie IP adresy kamerového modulu	40
14	Spustenie HTTP stream na ESP32-CAM	41
15	Posielanie UDP packetov	41
16	Vnútorňý cyklus Arduina	42

Úvod

Autonómne vozidlá sú v súčasnosti jedným z najrýchlejšie sa rozvíjajúcich technologických odvetví. Ide o formu mobility, ktorá je bezpečnejšia, efektívnejšia a pohodlnejšia než konvenčné spôsoby dopravy riadené človekom. Zavedenie autonómnych vozidiel vedie k presunu pracovnej sily od rutinných manuálnych úkonov smerom k činnostiam zameraným na monitorovanie, manažovanie a kontrolu systémov. Taktiež umožňujú ľuďom s fyzickým postihnutím samostatne operovať automobil. V súčasnosti sú vozidlá s čiastočnou autonómiou dostupné pre každého, kto túži po luxuse, ktorý tieto autá poskytujú, a má na to potrebné financie.

Autonómne vozidlá sa rozdeľujú do šiestich stupňov autonómie, od 0 (žiadna autonómia) až po 5 (úplná autonómia bez nutnosti zásahu človeka). Na nahradenie funkcií človeka využívajú senzory na vnímanie okolia a umelú inteligenciu na rozhodovanie. Technologické spoločnosti sa v tejto oblasti snažia o čo najefektívnejšie prepojenie týchto dvoch systémov.

Najväčšími bariérami pri integrácii autonómnych vozidiel do bežnej spoločnosti sú ich vysoká cena a neschopnosť navigovať v komplikovaných hraničných situáciách. Súčasný vývoj sa snaží tieto bariéry aktívne prekonávať.

Cielom bakalárskej práce je navrhnúť a implementovať riešenie autonómneho vozidla, ktoré je schopné detekovať prekážky a adekvátne na ne reagovať. Projekt realizujeme na platforme Arduino s využitím knižnice na počítačové videnie OpenCV. V rámci práce preskúmame aktuálne prístupy k autonómii a zvažíme životaschopnosť súčasných trendov. Výstupom bude praktická implementácia modelu vozidla a jeho testovanie v rôznych situáciách.

Výber témy bakalárskej práce vychádza z nášho záujmu o automatizáciu a inovácie v oblasti automobilov. Daná téma spája mnohé populárne technológie, ako je umelá inteligencia a automatizácia logistiky. Zároveň nás zaujala práca s mikrokontrolérmi a elektronikou. Práve kombináciou týchto oblastí sme sa pre túto tému nadchli.

1 Autonómne vozidlá

1.1 Čo je to autonómne vozidlo?

Autonómne vozidlo (AV), niekedy nazývané aj samoriadiace auto je dopravný prostriedok, ktorý, na rozdiel od tradičných vozidiel nevyžaduje vodiča. Autonómne vozidlá vykonávajú túto úlohu pomocou viacerých hardvérových a softvérových systémov. Hoci ani v roku 2026 nemá termín 'samoriadiace' jednotnú štandardnú definíciu, hlavným cieľom týchto technológií zostáva eliminácia ľudských chýb, zvýšenie efektivity premávky a sprístupnenie prepravy ľuďom, ktorí nie sú schopní šoférovať sami [1].

Na vnímanie okolia vozidla používame senzory detekcie objektov. Systém vnímania používa kamery, Radio Detection and Ranging (RADAR), Light Detection and Ranging (LiDAR) a ultrazvukové senzory, pričom každý z nich má špecifickú funkciu. Autonómne vozidlá často pracujú s čiastočne neznámym a dynamickým prostredím, preto si musia vystavať model svojho prostredia a zároveň sa v ňom lokalizovať. Tento proces je známy pod skratkou SLAM (súčasná lokalizácia a mapovanie) [1].

Spočíva v spracovaní dát zo senzorov, máp a systéme globálneho polohovacieho systému (GPS), vďaka čomu si zariadenie vytvorí model prostredia a určí v ňom svoju polohu v reálnom čase [1].

Tretí systém je zodpovedný za plánovanie a rozhodovanie. Na základe dát z predchádzajúcej vrstvy vozidlo vyhodnocuje a reaguje na vývoj dopravnej situácii. Táto vrstva má mnoho funkcií od plánovania trasy po vyhýbanie sa prekážkam. Najčastejšie sa používajú modely strojového učenia, vrátane neurónových sietí [1].

Posledným článkom je konanie a ovládanie. Systém ovládania vozidla premení abstraktné požiadavky predchádzajúcej vrstvy na fyzické pohyby auta. Zodpovedá za smerové riadenie, zrýchľovanie a brzdenie.

Pokročilé algoritmy zároveň zabezpečujú, aby pohyb bol plynulý, stabilný a rešpektoval limitácie podvozku. Vďaka integrácii týchto systémov dokáže autonómne vozidlo navigovať v komplexných dopravných situáciách s minimálnou až nulovou potrebou ľudského zásahu [1].

1.2 Úrovne autonómnych vozidiel

Úrovne autonómnych vozidiel sú definované organizáciou SAE International v štandarde SAEJ3016. Tento dokument delí automatizáciu riadenia do šiestich stupňov (0–5) podľa toho, akú časť jazdy zabezpečuje systém a akú úlohu ešte stále zohráva vodič. O samoriadiacich autách sa často hovorí, ako o jednej kategórii, no v praxi ide o postupný

vývoj od minimálnej podpory riadenia až po úplnú autonómiu vozidla bez zásahu človeka [2].

1.2.1 Úroveň 0

Žiadna automatizácia Na tejto úrovni všetky činnosti vykonáva ľudský vodič. Vozidlo môže mať bezpečnostné systémy, ako napríklad upozornenie opustenia jazdného pruhu, no samotné riadenie je plne v rukách človeka [2].

1.2.2 Úroveň 1

Asistované riadenie Systém pomáha s postranným alebo pozdĺžnym riadením vozidla ale nie v obidvoch prípadoch naraz. Sem patria asistencie, ako udržiavanie rýchlosti (tempomat) alebo mierne korigovanie smeru. Vodič však musí neustále sledovať situáciu na vozovke a mať ruky na volante [2].

1.2.3 Úroveň 2

Čiastočná automatizácia Vozidlo dokáže súčasne riadiť smer aj rýchlosť, stále však platí, že vodič nepretržite dohliada na jazdu a je pripravený zasiahnuť v prípade núdze. Systém vykonáva viac úloh riadenia naraz, no zodpovednosť upadá na človeka [2].

1.2.4 Úroveň 3

Podmienená automatizácia Jazda v špecifickej situácii je automatizovaná a vodič nemusí aktívne sledovať okolie. Ak však systém vyhodnotí, že situáciu nezvládne, kontrola vozidla sa vráti vodičovi. Vodič musí byť pripravený prevziať kontrolu nad vozidlom [2].

1.2.5 Úroveň 4

Vysoká automatizácia Na tejto úrovni je vozidlo schopné zvládnuť jazdu úplne samo v rámci definovaných prostredí alebo podmienok. V prípade, že sa vozidlo dostane mimo týchto podmienok, dokáže bezpečne zastaviť. Prítomnosť ľudského šoféra nie je nutná [2].

1.2.6 Úroveň 5

Plná automatizácia Najvyšší stupeň automatizácie, vozidlo dokáže jazdiť samostatne za všetkých podmienok, v ktorých dokáže jazdiť človek. Neexistuje potreba vodiča. Vozidlo zastaví v prípade zlyhania určitých modulov riadenia [2].

2 Ciele práce

V tejto kapitole si stanovíme hlavný cieľ bakalárskej práce a taktiež čiastkové ciele, ktoré slúžia, ako milníky pri praktickom spracovaní témy práce.

Hlavný cieľ bakalárskej práce je návrh a implementácia algoritmu pre autonómne riadenie malého robotického vozidla, ktoré dokáže v reálnom čase detegovať prekážky v jazdnej dráhe a následne na ne reagovať.

Praktická časť práce sa zameriava na implementáciu riešenia na hardvérovej platforme Arduino so súčasnými metódami počítačového videnia pomocou knižnice OpenCV.

2.1 Čiastkové ciele

1. **Analýza problematiky:** Teoretické spracovanie aktuálnej problematiky v oblasti autonómnych vozidiel. Zhodnotenie súčasnej a budúcej implementácie jednotlivých systémov použitých v AV. Prehľad konkrétneho využitia senzorových systémov a umelej inteligencie na detekciu prekážok.
2. **Návrh technologického riešenia:** Výber optimálnej kombinácie hardvérových a softvérových komponentov v rámci obmedzenia bakalárskej práce. Zvolenie riadiacej jednotky Arduino, typ a počet senzorov, programovací jazyk riešenia a mechanické prvky vozidla. Výber metód detekcie prekážky a jazdného pruhu pomocou počítačového videnia. Fyzický návrh vzhľadu vozidla so zameraním na použité materiály a rozloženie komponentov.
3. **Algoritmizácia detekcie:** Zvolenie riešení v prostredí OpenCV zameraných na spracovanie obrazu z kamery na identifikáciu prekážok a jazdného pruhu v zornom poli vozidla.
4. **Prepojenie komponentov:** Návrh metód komunikácie jednotlivých komponentov. Prepojenie vrstiev riadenia autonómneho vozidla od vnímania po konanie. Interpretácia výstupov na vstupy pre jednotlivé systémy. Návrh fyzického prepojenie častí systému pomocou komunikačného média.
5. **Zostavenie riešenia:** Implementácia zvolených návrhov. Naprogramovanie riešenia s použitím knižnice OpenCV. Zostavenie rámu vozidla a následná montáž.
6. **Testovanie a validácia:** Navrhnutie metodiky testovania a následné overenie funkčnosti vozidla v rozličných situáciách. Otestovanie detekcie rôznych prekážok.

3 Analýza súčasného stavu

3.1 Aktuálny stav technológie

V posledných rokoch sme zaznamenali nárast vozidiel s tretou úrovňou autonómie, autonómne taxi služby sa v mestách stávajú reálnym spôsobom dopravy a videli sme už aj prvé ukážky kamiónov bez nutnosti ľudského vodiča [3][4].

Vďaka vývoju v oblasti umelej inteligencie a zvýšeniu výkonu výpočtových platforiem sme zaregistrovali značný pokrok v technológiách autonómnych vozidiel. Výrobcovia sa snažia zlepšovať kombinácie radarových, LiDARových a kamerových dát tak, aby boli robustnejšie voči dopravným a poveternostným podmienkam. Pokračuje snaha v návrhu scenárov pre hraničné situácie, čím sa zlepší percepčná schopnosť autonómnych systémov a zrýchli ich nasadenie [5][6].

Autonómne vozidla zaznamenali značnú investíciu ale aj kontrolu zo strany štátnych a nadnárodných organizácií. Spoločnosti, ktoré poskytujú služby samoriadiacich vozidiel rozširujú svoju dostupnosť, znižujú počet senzorov a náklady na hardvér. Hoci technologický pokrok napreduje, integrácia do infraštruktúry mesta zostáva podstatnou výzvou pre adopciu AV do dopravy. Štandard SAE International preto opisuje plán prechodu od asistencie šoférovanie po plne autonómnu prevádzku [7][8].

3.1.1 Umelá inteligencia

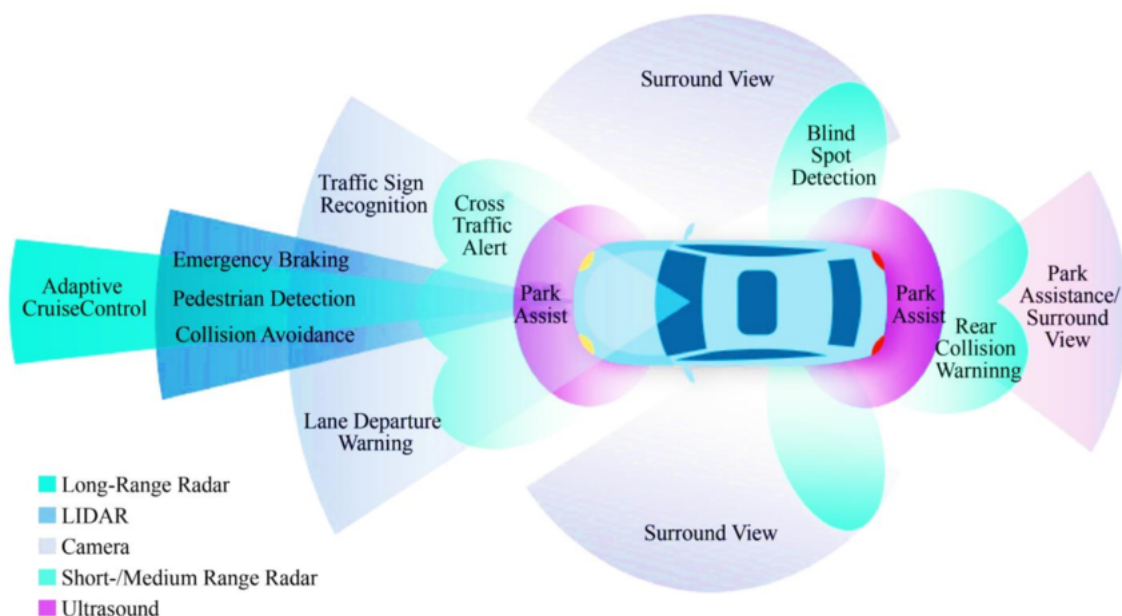
Umelá inteligencia (AI) vykonáva dôležitú funkciu v percepčnom, rozhodovacom a plánovacom systéme autonómneho vozidla, čím nahrádza rolu ľudského vodiča. Moderné systémy používajú strojové učenie (machine learning) na detekciu hazardov na ceste, klasifikáciu dopravných značiek a sledovanie a predikciu pohybu chodcov a ostatných vozidiel [9].

V súčasnosti sa autonómne modely trénujú na obrovskom množstve anotovaných obrázkov, videí, simulácií a skutočných jazdných dát, aby v praxi zvládli širokú škálu scenárov. Hlboké učenie (deep learning) a učenie posilňovaním (reinforcement learning) sa používajú v novodobých autonómnych vozidlách. Deep learning, podskupina strojového učenia, pracuje na princípe viacvrstvových neurónových sietí, čím sa fungovaním podobá ľudskému mozgu. Reinforcement learning mení správanie modelu využitím systému odmien a penalizácií [10].

Napriek značnej komplexnosti zostáva riešenie hraničných prípadov (edge cases) zásadnou výzvou pre dnešné systémy. Potreba veľkého množstva tréningových dát a vysoká hranica bezpečnosti a spoľahlivosti výrazne spomaľuje nasadenie vozidiel do reálnej premávky [6].

3.1.2 Senzory

Senzory sú spôsob akým autonómne vozidlá vnímajú svoje okolie. Ako je možné vidieť na obrázku 1, v súčasnosti sa v autách využíva kombinácia kamier, LiDARových, radarových a ultrazvukových senzorov [11].



Obr. 1: Ako AV vnímajú prostredie

[9], licencia: CC BY-SA 4.0.

Kamery predstavujú najrozšírenejší typ senzorov; sú nákladovo efektívne, no citlivé na zlé osvetlenie a počasie. Používajú sa na detekciu jazdných pruhov, rozpoznávanie dopravných značiek a identifikáciu chodcov a vozidiel. RADAR využíva elektromagnetické vlny na meranie vzdialenosti a rýchlosti. Používa sa najmä pre detekciu vzdialených objektov aj v nepriaznivých poveternostných podmienkach (dážď, hmla, sneh). LiDAR používa laserové impulzy na vytvorenie detailného trojrozmerného modelu okolia, no okrem vysokých nákladov je aj viac náchylný na zlé počasie. Ultrazvukové senzory majú využitie v detekcii objektov na krátke vzdialenosti, napríklad pri parkovacích manévroch. Na rozdiel od fotoelektrických senzorov (napr. RADAR), nie sú ovplyvnené farbou alebo odrazivosťou cieľa, pretože používajú zvuk a nie svetlo [1][11].

Súčasný vývoj v oblasti senzorov sa zameriava primárne na zvyšovanie presnosti meranie, optimalizovanie počtu potrebných senzorov a výrazné znižovanie ich ceny. Výrobcovia taktiež zefektívňujú kombinovanie dát viacerých senzorov, tzv. senzorovú fúziu, čím zvyšujú presnosť a spoľahlivosť systému [8][9].

3.2 Použitie autonómnych vozidiel

Autonómne vozidlá ponúkajú široké spektrum využitia, pričom ich možnosti sa v posledných rokoch rozširujú. K najpozoruhodnejším oblastiam ich využitia patrí doprava. Hlavným cieľom je zvýšiť bezpečnosť a plynulosť premávky. Ďalšie využitie je v logistike tovaru, kde nahrádzajú rutinnú manuálnu prácu tradične vykonávanú ľuďmi.

3.2.1 Autonómne taxi služby

Autonómne taxi služby, často nazývané robotaxíky, predstavujú jednu z najpokročilejších foriem využitia autonómnych technológií. Celý proces – od objednania a naplánovania trasy až po samotnú prepravu pasažierov – je plne automatizovaný.

Mnohí výrobcovia automobilov, technologické spoločnosti a štátne organizácie investujú zdroje do vývoja robotaxíkov. Na popredí stojí spoločnosť Waymo, dcérska spoločnosť amerického konglomerátu Alphabet. Ich vozidlá, splňajúce štvrtú úroveň autonómie, využívajú LiDAR, RADAR a kamery na vytváranie trojrozmerného obrazu okolia. S každou ďalšou generáciou vozidiel navyše redukovujú počet senzorov, čím znižujú výrobné náklady a zlepšujú škálovateľnosť celého projektu [8].

Alternatívny prístup zvolila spoločnosť Tesla so svojou službou Robotaxi a systémom FSD (Full Self-Driving), ktorý predstavuje tretí stupeň autonómneho riadenia. Na rozdiel od konkurencie nepoužíva finančne nákladné LiDAR senzory, ale spolieha sa výhradne na kamerový systém s interpretáciou dát pomocou umelej inteligencie. Hoci pri bežných vozidlách nad jazdou stále dohliada ľudský operátor pripravený kedykoľvek prevziať kontrolu, v roku 2026 začala spoločnosť do svojej flotily zavádzať prvé vozidlá v plne autonómnom režime bez ľudského dozoru [12].

Ďalším významným hráčom je spoločnosť Nvidia s modelom AlpaMayo 1. Tento open-source model uvažovania je navrhnutý a trénovaný špeciálne na zvládanie komplexných a kritických dopravných situácií. Vydanie tohto voľne dostupného riešenia má za cieľ výrazne urýchliť integráciu štvrtej a piatej úrovne AV do bežnej premávky [6].

3.2.2 Autonomizácia donášky jedla

Autonómne doručovacie roboty (ADR) sa používajú v súčasnosti v univerzitných kampusoch a mestách na donášku jedla. Poskytujú takzvanú last-mile delivery (dodanie na poslednú míľu) a fungujú na štvrtej úrovni autonómie. Primárne využívané na chodníkoch alebo cyklochodníkoch, doručovacie roboty sú schopné prekonávať križovatky, vyhýbať sa prekážkam a doručovať tovar na vzdialenosť pár kilometrov [13].

ADR používajú zväčša kombináciu kamier a počítačového videnia na navigovanie, týmto spôsobom zabezpečujú nižšie náklady na výrobu, s väčšou mierou škálovateľnosti.

Niektorí výrobcovia sa vyhýbajú používaniu GPS, pretože neposkytuje dostatočnú mieru presnosti [14].

Autonómna donáška jedla zaznamenala výrazný rozvoj počas pandémie COVID-19. Mnohé spoločnosti poskytujúce donášku jedla, ako napríklad Uber Eats a DoorDash, ale aj logistické spoločnosti, ako Amazon, Walmart a Google začali investovať do vývoja ADR. Jedným z priekopníkov v tejto oblasti je spoločnosť Starship Technologies, ktorá výrazne rozšírila svoju flotilu práve počas pandémie. Zatiaľ čo donáška pomocou pozemných robotov je bežná, mnohé spoločnosti investujú aj do vývoja doručovania pomocou autonómnych dronov [14].

Značným problémom ADR je ich krátky dosah, pomalá rýchlosť a obmedzená kapacita užitočného zaťaženia. Mnohé spoločnosti investujú do hybridných modelov donášky kombináciou konvenčných donáškových vozidiel a doručovacích robotov [13].

V závislosti od charakteristík systému možno existujúci model hybridného doručovania klasifikovať, ako nasledujúce tri kategórie:

- **Dvojúrovňový model:** Tento model využíva konvenčné vozidlá (kamióny alebo dodávky) na hromadnú prepravu zásielok z centrálného depa do siete menších lokálnych skladísk. Z týchto miest vykonávajú ADR donášku na poslednú míľu priamo zákazníkovi [13].
- **Mothership model:** V tomto modeli slúži štandardné doručovacie vozidlo ako materská loď, ktorá je špeciálne upravená na prepravu flotily ADR. Autonómne roboty sa z materskej lode vypúšťajú v bezprostrednej blízkosti koncových adries doručenia. Ide o jeden z najčastejšie skúmaných modelov v aktuálnej literatúre [13].
- **Model čaty** Autonómne vozidlá doručujú zásielky samostatne v oblastiach, ktoré sú schopné navigovať. Ak však musia prekonať zóny, ktoré sú pre roboty nedostupné, pripoja sa k manuálne riadenému vozidlu, ktoré funguje ako „vodca čaty“. Roboty nasledujú toto vozidlo, až kým nedorazia do zóny im vhodnej [13].

3.3 Zhrnutie poznatkov

Po analýze technologického vývoja autonómnych vozidiel je nevyhnutné kriticky zhodnotiť ich dopad na reálne dopravné prostredie. V nasledujúcej podkapitole si identifikujeme aktuálne trendy a vyvodíme poznatky, ktoré formujú súčasnú sféru autonómnej mobility.

Mnohé implementácie v priemysle autonómnych vozidiel sa posúvajú od asistovaných systémov úrovne 2 a 3, k plne autonómnej prevádzke v špecifických podmienkach úrovne 4. Plná autonómia úrovne 5 stále ostáva dlhodobým problémom v danej oblasti.

Počet senzorov sa v autonómnych vozidlách zdatne znižuje. Prechádza sa z ideológie „maximálnej redundancie“ s vysokým počtom senzorov na optimalizovaný prístup. Spoločnosti, ako Tesla nahrádzajú spoľahlivé, no ekonomicky nevýhodné LiDARové senzory za kombináciu kamier a AI.

Kontrastne, spoločnosti ako Waymo sa držia prístupu založeného na LiDARe, no snažia sa znížiť cenu a zvýšiť ich efektivitu. V súčasnosti ešte neexistuje konsenzus medzi technologickými spoločnosťami.

AI zohráva dôležitú rolu pri plánovaní a rozhodovaní systéme autonómnych vozidiel. Súčasný trend ukazuje na využívanie jazykových modelov, ktoré sú schopné interpretovať dáta zo senzorov a z nich priamo rozhodnúť o chode autonómneho vozidla.

Hraničné situácie zohrávajú najväčšiu bariéru ku masovému nasadeniu autonómnych vozidiel úrovne 5. Vývoj v oblasti umelej inteligencie sa zameriava práve na tieto edge cases. Z analýzy vyplýva, že vyriešením tohto problému, bude zavedenie AV v mestách zdatne urýchlené.

Technológia samotná nestačí pre adopciu autonómnych vozidiel do miest. Značným problémom je integrácia v mestskom prostredí a verejná akceptácia AV. Predpokladáme, že krajiny, ktoré budú investovať do integrácie autonómnych vozidiel, a ktoré majú progresívny názor na ich nasadenie, budú zaznamenávať najväčší nárast v tomto sektore.

Viacero z identifikovaných aspektov boli priamo reflektované v našom návrhu systému autonómneho vozidla s detekciou prekážok.

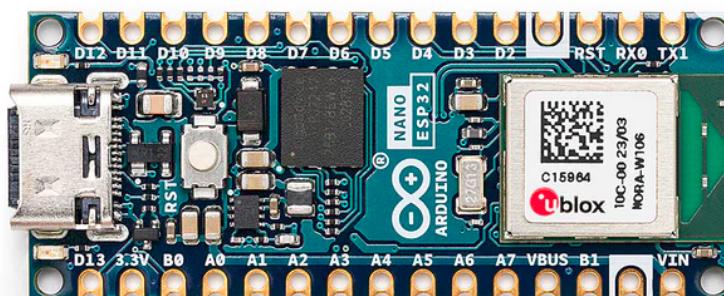
4 Komponenty systému

V tejto kapitole si preberieme použité komponenty celkového systému. Výber správnych súčastí v rámci limitácií bakalárskej práce je dôležitý pre získanie lepších výsledkov.

4.1 Arduino Nano ESP32 mikrokontrolér

Arduino je talianska open-source hardvérová a softvérová spoločnosť, ktorá navrhuje a vyrába mikrokontroléry určené na vývoj rôznych zariadení. Rodina mikrokontrolérov Arduino Nano predstavuje radu kompaktných a cenovo dostupných dosiek. Konkrétne model Nano ESP32 je osadený čipom ESP32-S3, ktorý integruje možnosti Wi-Fi a Bluetooth bezdrôtového pripojenia. Tento mikrokontrolér poskytuje optimálny pomer veľkosti a výkonu, s dostatočným počtom pinov na pripojenie periférií [15].

V tejto práci sa Arduino Nano ESP32 používa ako riadiaca jednotka, ktorá získava vstupy od ostatných komponentov a podľa nich riadi mechanické časti vozidla [15].



Obr. 2: Arduino Nano ESP32

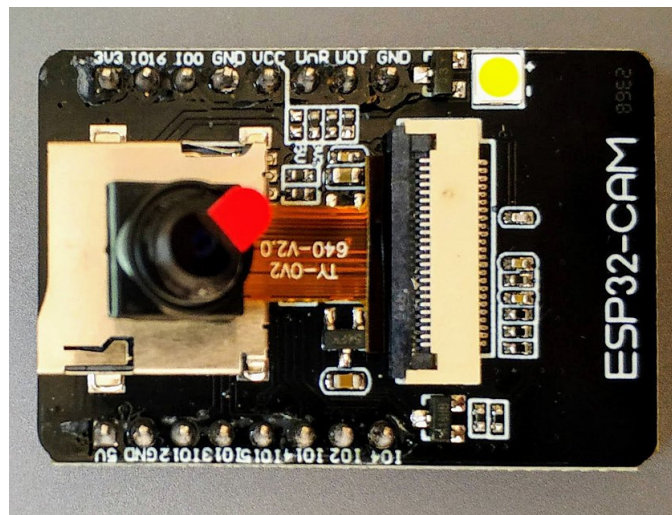
autor: Arduino, zdroj: arduino.cc, licencia: CC BY-SA 4.0.

Technické parametre:

- Rozmery: 18 * 45 mm
- Vstupné/výstupné napätie: 3,3 V
- Počet GPIO pinov: 48
- Komunikačné rozhrania: UART, I2C, SPI
- Rýchlosť procesora: 240 Mhz
- Veľkosť ROM: 384 kB
- Veľkosť SRAM: 512 kB
- Veľkosť RAM: 8 MB

4.2 ESP32-CAM + OV2640 kamera

Vývojová doska ESP32-CAM integruje Wi-Fi a Bluetooth modul spolu s rozhraním na pripojenie 2 Mpx kamerového senzora OV2640. Táto kombinácia poskytuje oddelenú implementáciu prenosu videa, čím sa odľahčí výpočtová záťaž hlavného čipu. Senzor OV2640 disponuje natívnym rozlíšením 1600 x 1200 pixelov a snímkovou frekvenciou 60 FPS, čo poskytuje dostatočnú výkonovú rezervu [16][17].



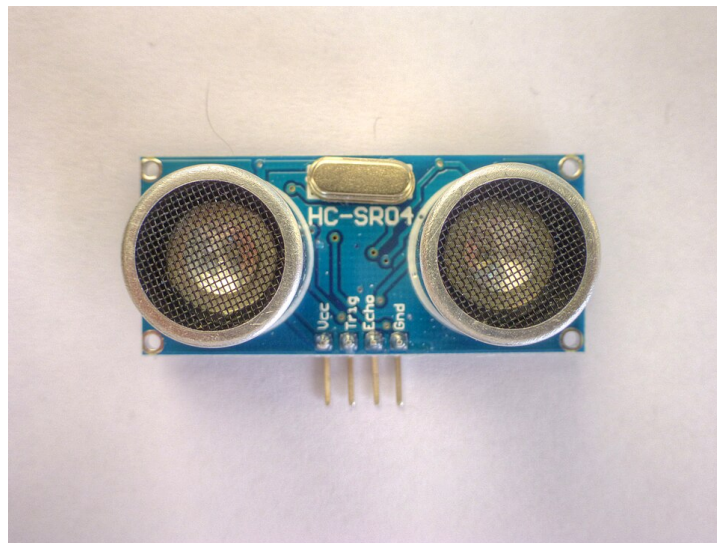
Obr. 3: ESP32-CAM a OV2640
ESP32-CAM, autor: Nowforever, zdroj: Wikimedia
Commons, licencia: CC BY-SA 4.0.

Technické parametre:

- Rozmery: 27 * 40 mm
- Pracovné napätie: 2,2 - 3,6 V
- Rozlíšenie kamery: 1 600 x 1 200
- Frame rate kamery: 60 FPS (pri CIF formáte)
- Rýchlosť procesora: 240 Mhz
- Veľkosť FLASH: 4 MB
- Veľkosť PSRAM: 4 MB

4.3 HC-SR04 Ultrazvukový senzor

Ultrazvukový senzor HC-SR04 predstavuje nákladovo efektívne riešenie na detekciu objektov pred vozidlom. Tento senzor slúži ako sekundárny systém pre núdzové zastavenie v prípadoch, kedy primárny kamerový systém zlyhá pri detekcii prekážky [18].



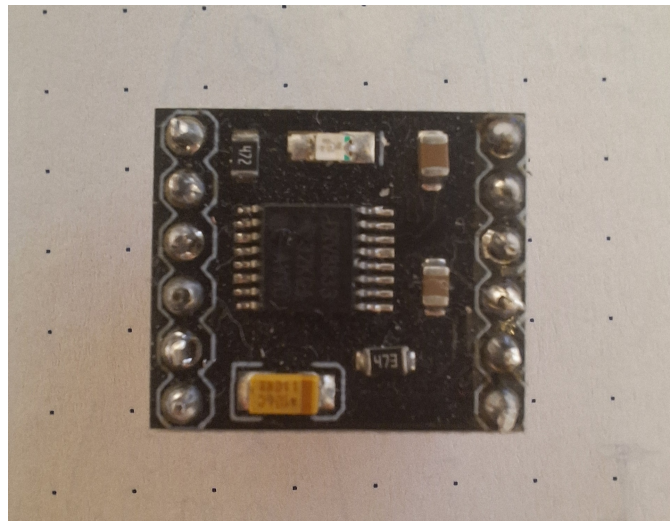
Obr. 4: HC-SR04 ultrazvukový modul
HC SR04 Ultrasonic Module, autor: Nevit Dilmen, zdroj:
Wikimedia Commons, licencia: CC BY-SA 3.0.

Technické parametre:

- Rozmery: 20 * 45 * 15 mm
- Pracovné napätie: 5 V
- Maximálna vzdialenosť: 4 m
- Minimálna vzdialenosť: 2 cm
- Merací úhol: 15°

4.4 DRV8833 driver pre motorčky

Modul DRV8833 bol použitý na nezávislé riadenie dvoch DC motorov. Integrovaný obvod obsahuje dvojité H-mostík, ktorý umožňuje meniť polaritu napätia, a tým aj smer otáčania motorov. Modul má štyri vstupné piny (dva pre každý motor), pomocou ktorých možno ovládať rýchlosť a smer otáčania [19].



Obr. 5: DRV8833 driver pre motorčky

Technické parametre:

- Rozmery: 5 * 4 mm
- Logické napätie: 2,7 - 10,8 V
- Drive napätie: 3 - 10 V
- Drive prúd: 1,5 A (MAX pre jeden mostík)
- Maximálny výkon: 15 W

4.5 Ďalšie elektrické komponenty

V implementácii je použité 18650 Li-Ion akumulátor s napätím 7,4 V a kapacitou 3 000 mAh.

Pohon zabezpečujú dva GA12-N20 DC motory s prevodovkou s rýchlosťou 200 RPM pri napätí 3 V.

Z dôvodu ochrany komponentov je do systému zapojený znižujúci menič napätia (step-down buck converter). Použitý menič disponuje dvojvodičovým vstupom a výstupom v podobe dvoch konektorov USB-A. Zabudovaný LM2596 čip reguluje široké pásmo napätí na 5 V s výstupným prúdom 3 A.

Súčasťou zapojenia je taktiež trojpólový, dvojpohový MTS-102 prepínač na zapnutie a vypnutie celého systému.

Prepojenie jednotlivých komponentov zabezpečuje nepájivé pole a spojovacie vodiče.

4.6 Server

Vzhľadom na obmedzený výpočtový výkon mikrokontrolérov Arduino, ktoré nie sú schopné vykonávať komplexné výpočty neurónových sietí v reálnom čase, sme do systému zaradili centrálny server. Ten zabezpečuje spracovanie obrazu a koordináciu bezdrôtových periférií. V našej implementácii túto úlohu plní laptop, ktorý zároveň slúži ako prístupový bod pre Wi-Fi sieť.

5 Softvér

V nasledovnej časti sa oboznámime s použitým softvérom, ktorý nám umožňuje preklad požiadaviek systému na skutočné inštrukcie vykonateľné hardvérom.

5.1 C++

C++ je programovací jazyk vyššej úrovne na všeobecné použitie, ktorý zároveň umožňuje prácu s prostriedkami nízkej úrovne. Bjarne Stroustrup vyvinul C++ v roku 1983 ako rozšírenie jazyka C [20].

Mikrokontroléry Arduino sa programujú v jazykoch C/C++ pomocou štandardného rozhrania pre programovanie aplikácií (API), ktoré je známe ako Arduino Programming Language. Toto API poskytuje funkcie a knižnice určené na ovládanie hardvéru. Súčasťou projektu Arduino je aj vývojové prostredie Arduino IDE, ktoré zjednodušuje kompiláciu, integráciu knižníc a samotné nahrávanie kódu [15].

V našej implementácii používame C++ pre programovanie nízkoúrovňového mikrokontroléru Arduino Nano ESP32 a kamerového modulu ESP32-CAM.

5.2 Python

Python je interpretovaný, interaktívny programovací jazyk vysokej úrovne, ktorý vytvoril Guido van Rossum. Je dynamicky typovaný, čo znamená, že typ premennej sa určuje za behu programu. Na rozdiel od jazykov C/C++, využíva Garbage Collection, vďaka čomu programátor nemusí manuálne riešiť alokovanie a uvoľňovanie pamäte. Štruktúra kódu je definovaná odsadením, čím sa výrazne zvyšuje čitateľnosť kódu [21].

V našej implementácii používame Python na vytvorenie Wi-Fi servera, spracovanie dát z kamery a následnú interpretáciu výsledkov do podoby príkazov pre pohyb kolies.

5.3 OpenCV

OpenCV je knižnica programovacích funkcií určených na počítačové videnie v reálnom čase, ktorú pôvodne vyvinula spoločnosť Intel. Využíva sa v oblastiach detekcie objektov, tváří a gest, pri vizuálnej odometrii, vnímaní hĺbky či v augmentovanej realite. Knižnica OpenCV je napísaná v programovacom jazyku C++, no ponúka rozhrania pre integráciu v ďalších jazykoch, ako sú Python, Java alebo MATLAB [22][23].

V našej implementácii používame OpenCV na detekciu vozovky a analýzu jazdných pruhov.

5.4 NumPy

NumPy je knižnica pre programovací jazyk Python, ktorá pridáva podporu pre optimalizované polia, matice a matematické funkcie, ktoré s nimi pracujú [24].

Python interpretuje inštrukcie riadok po riadku, čo spomaľuje celkový program. Knižnica NumPy urýchľuje tento proces tým, že vykonáva výpočtovo náročné operácie vo vysoko optimalizovanom, kompilovanom C kóde. Taktiež mnoho operácií uvoľňujú globálny zámok interpreteru, čím dovoľuje paralelné spúšťanie viacerých vlákien viacjadrového procesora. Horeuvedená knižnica OpenCV používa NumPy na ukladanie dát a operácie s nimi [23][24].

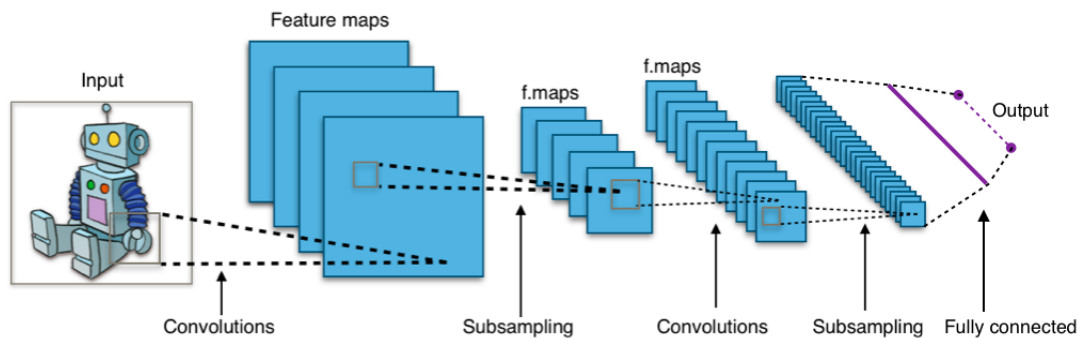
V našej implementácii používame knižnicu NumPy na prácu s vektormi a maticami pri transformácii obrazových dát.

5.5 CNN

Convolutional Neural Network (CNN), po slovensky konvolučná neurónová sieť, je typ doprednej neurónovej siete, ktorá používa optimalizáciu filtrov. Tento typ hlbkej neurónovej siete sa štandardne používa pri počítačovom videní a spracovaní obrazu [25].

Konvolučné neurónové siete sa skladajú z rôznych vrstiev. Hlavným stavebným prvkom je takzvaná konvolučná vrstva, ktorá obsahuje rôzne filtre – matice váh. Počas doprednej propagácie sa po celej výške a šírke vstupnej matice počíta skalárny súčin pre každý filter konvolučnej vrstvy. Výsledkom je dvojrozmerná aktivačná mapa daného filtra. CNN sa učia vytvárať filtre, ktoré sa aktivujú v prípade, že sa na vstupe nachádza požadovaná vlastnosť [25].

Ďalšie vrstvy, ako združovacia (pooling) vrstva sa používa na zmenšenie rozmerov vstupnej matice, ReLU vrstva na odstránenie záporných hodnôt a plne prepojená (fully connected) vrstva na finálnu klasifikáciu. Obrázok 6 zobrazuje bežnú konvolučnú neurónovú sieť [25].



Obr. 6: Typická CNN

Fully connected convolutional neural network, autor:
Aphex34, zdroj: Wikimedia Commons, licencia: CC BY-SA
4.0.

CNN dosahujú pri analýze obrazového vstupu výrazne nižšiu chybovosť a efektívnejší proces učenia v porovnaní s inými typmi neurónových sietí. V súčasnosti sa konvolučné neurónové siete používajú na detekciu prekážok v autonómnych vozidlách, no objavuje sa rastúci trend prechodu na veľké jazykové modely (LLM), ktoré ovládajú všetky systémy vozidla [3][6][26].

5.5.1 Ultralytics YOLOv8

YOLOv8 je systém od spoločnosti Ultralytics určený na detekciu objektov v reálnom čase; funguje na základe konvolučných neurónových sietí. Metóda You Only Look Once (YOLO) vyžaduje iba jednu doprednú propagáciu (forward pass) na vytvorenie predpokladu, preto je ideálna pre aplikácie v reálnom čase [27][28].

YOLOv8 ponúka päť veľkostí modelov: nano, small, medium, large a extra-large, ktoré predstavujú kompromis medzi rýchlosťou a presnosťou. Tieto modely sú predtrénované na rozsiahlych datasetoch, vďaka čomu sú pripravené na okamžité nasadenie bez potreby zložitej konfigurácie [28].

V našej implementácii využívame veľkosť modelu *small* na detekciu prekážky v dráhe vozidla.

6 Návrh a implementácia

Správny návrh architektúry je dôležitý pre efektívne fungovanie všetkých komponentov. Počas vývoja bolo otestovaných niekoľko prístupov, kým sa nepodarilo dosiahnuť optimálne riešenie. V tejto kapitole popíšeme použitý dizajn a jeho následnú implementáciu.

6.1 Detekcia vozovky

Detekcia jazdných pruhov a hraníc vozovky predstavuje jeden z kritických subsystémov autonómnych vozidiel. Jej hlavnou úlohou je identifikovať cestu, udržať vozidlo v jazdnom pruhu a správne zareagovať v prípade zmien vozovky. AV sú schopné túto funkciu vykonávať pomocou fúzie rôznych senzorov [11].

Táto podkapitola sa zaoberá metódami spracovania dát, ktoré umožňujú systému rozoznať vozovku od prostredia. Na tento účel používame metódy a funkcie dostupné v programovacom jazyku Python, ako aj v knižniciach NumPy a OpenCV. Tieto knižnice boli do vývojového prostredia nainštalované pomocou správcu balíkov pip.

6.1.1 Region of interest

V kontexte autonómnych vozidiel sa ROI používa na ohraničenie určitej oblasti obrazového vstupu kamier, ktorá je pre daný systém detekcie relevantná [23].

Pri implementácii detekcie vozovky bol použitý ROI ako prvý krok predspracovania. Vozovka sa vo väčšine prípadov nachádza v spodnej polovici obrazu, pričom spracovanie hornej časti zbytočne vnáša nepotrebné informácie. To spôsobuje vizuálny šum a zaťaženie procesora.

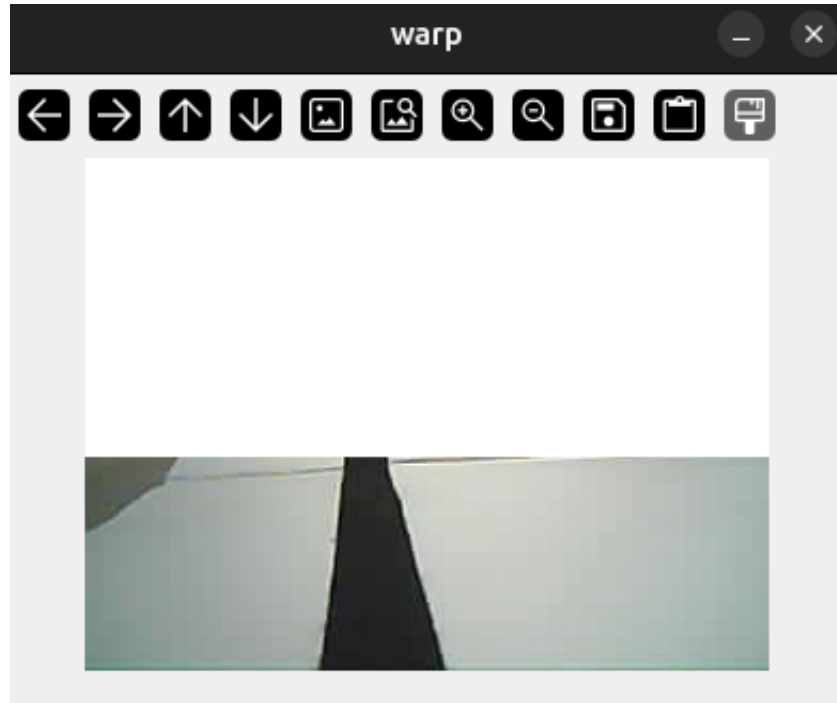
Na extrakciu ROI boli použité funkcie dostupné v knižnici OpenCV.

```
1 def region_of_interest(frame):
2     h, w = frame.shape[:2]
3     roi_height = 100
4     roi_mask = np.ones_like(frame) * 255
5
6     roi_mask[h - roi_height:h, :] = frame[h - roi_height:h, :]
7
8     return roi_mask
```

Výpis kódu 1: Funkcia ROI

Proces začal vytvorením pomocnej matice s rovnakými rozmermi ako vstupný obraz, pričom všetky jej prvky boli nastavené na maximálnu hodnotu 8-bitového rozsahu (255). Táto matica bola definovaná pomocou knižnice NumPy. Následne do nej bola skopírovaná relevantná časť obrazu od vopred definovanej vertikálnej hranice `roi_height` smerom

nadol. Výsledok, viditeľný na obrázku 7, predstavuje obraz, kde sú spracované len tie pixely, ktoré sú v spodnej polovici obrazu a pravdepodobne obsahujú informácie o vozovke.



Obr. 7: Obraz s použitím ROI

6.1.2 Hľadanie tmavých regiónov

Hľadanie tmavých regiónov na vozovke je jednoduchý a výpočtovo nenáročný spôsob detekcie jazdných pruhov. Autonómne vozidlá dokážu detegovať svetlý jazdný pruh na tmavom asfalte vďaka kontrastu týchto dvoch plôch. V princípe je použitý rovnaký prístup, no v tejto implementácii sa hľadá čierny pruh v strede bielej vozovky [23].

Prvým krokom je definícia hraníc pre najsvetlejší a najtmavší tmavý región. V kóde sú tieto hranice definované ako NumPy pole s reprezentáciou vo formáte odtieň-sýtosť-jas (HSV). Odtieň (hue) predstavuje hodnotu od 0 po 179 – čiže polohu na farebnom kolese. Sýtosť (saturation) je percentuálne vyjadrenie „čistoty” farby, teda množstva sivej farby, kde sa volia hodnoty od 0 po 255. Jas (value) určuje intenzitu svetla, kde hodnota 0 predstavuje absolútnu čiernu, zatiaľ čo hodnota 255 zodpovedá maximálnemu jasovi farby [23].

```
1 lower_black = np.array([0, 0, 0])  
2 upper_black = np.array([180, 255, 70])
```

Výpis kódu 2: Definície maxima a minima tmavých regiónov

V ďalšom kroku sa konvertuje obraz z kamery z pôvodného formátu BGR (blue-green-red) na HSV reprezentáciu pomocou OpenCV funkcie `cvtColor` s argumentom `COLOR_BGR2HSV`. Volaním funkcie `inRange` s vopred definovanými hranicami sa vygeneruje binárna maska pixelov, ktoré spadajú do tohto rozsahu [23].

Výstup obsahuje šum a diery, ktoré môžu viesť k nesprávnym výsledkom, preto je ich nutné eliminovať. Tento efekt je možné dosiahnuť pomocou morfológických transformácií [23].

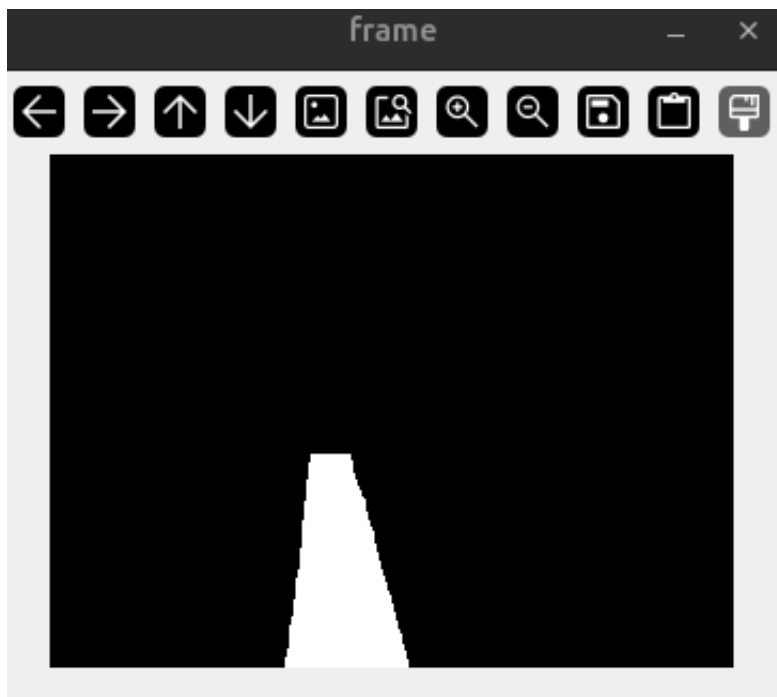
Šum predstavuje malé izolované regióny pixelov, ktoré do obrazu nepatria. Morfológické otvorenie dokáže tento šum odstrániť – v kóde je to realizované volaním funkcie `morphologyEx` s argumentom `MORPH_OPEN` a maticou jednotiek, ktorá funguje ako filter [23].

Na uzavretie dier – prazdnych miest vo vnútri objektu – sa používa rovnaká funkcia, no v tomto prípade s argumentom `MORPH_CLOSE`, čo predstavuje morfológické uzavretie [23].

```
1 def mask_road(frame):
2     hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
3     mask = cv.inRange(hsv, lower_black, upper_black)
4
5     kernel = np.ones((5, 5), np.uint8)
6     mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
7     mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
8
9     return mask
```

Výpis kódu 3: Maskovanie tmavých regiónov

Výstupné maskovanie silne závisí od externých faktorov, ako napríklad okolité osvetlenie. Obrázok 8 zobrazuje možný výstup maskovania.



Obr. 8: Hľadanie tmavého jazdného pruhu.

6.1.3 Hľadanie kontúr vozovky

Kontúry sú dôležitou súčasťou detekcie jazdných pruhov na vozovke. Princípom sa podobá hľadaním hranice tmavého objektu na svetlom pozadí [23].

V knižnici OpenCV existuje funkcia `findContours` na algoritmickú detekciu kontúr v obraze. Funkcia vracia dve polia — výstupné pole kontúr a pole opisujúce hierarchiu topológie obrazu. Táto štruktúra sa však v ďalšom procese ignoruje, keďže pre danú aplikáciu nie je relevantná [23].

Funkcia sa volá s dvomi parametrami. Prvým parametrom je režim vyhľadávania kontúr; využíva sa mód `RETR_EXTERNAL`, ktorý zachytáva iba najpovrchovejšie vonkajšie kontúry. Alternatívne režimy zachytávajú všetky dostupné kontúry, no líšia sa v ich hierarchickom usporiadaní — môže ísť o zoznam, dvojstupňovú alebo stromovú štruktúru [23].

Druhý parameter predstavuje algoritmus aproximácie kontúr. V implementácii je využívaný režim `CHAIN_APPROX_SIMPLE`. Z praktického hľadiska tento režim komprimuje segmenty bodov ležiace v jednej priamke — horizontálne, vertikálne alebo diagonálne — a ponecháva iba ich koncové body. To znamená, že štvorcová kontúra je zjednodušená iba na jej rohové body. Alternatívna metóda využíva algoritmus aproximácie pomocou Teh-Chinových reťazcov, zatiaľ čo iná ponecháva všetky body kontúry bez zmeny [23].

Implementácia kódu v systéme detekcie čiar vyzerá nasledovne.

```

1 def find_contours(mask):
2     contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
3     return contours

```

Výpis kódu 4: Detekcia kontúr

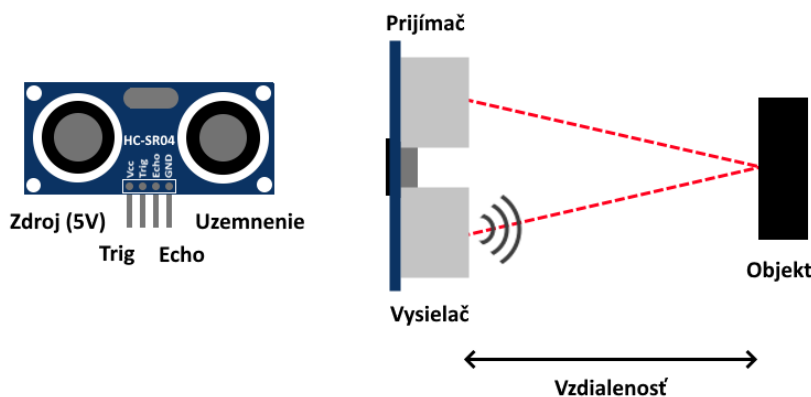
6.2 Detekcia prekážok

Dôležitým pilierom bezpečnosti autonómnych vozidiel je schopnosť detekcie prekážok. V reálnom čase musí systém identifikovať a klasifikovať objekty na vozovke alebo v jej blízkosti, ako aj adekvátne na ne reagovať. Tento proces využíva dáta z rôznych senzorov na vyhodnotenie situácie. Na základe týchto dát dokáže AV objekt obísť alebo pred ním bezpečne zastaviť [11].

V tejto podkapitole si vysvetlíme použité metódy na detekciu prekážok.

6.2.1 Ultrazvukový senzor

Pri tejto implementácii sme použili ultrazvukový senzor na detekciu prekážok priamo pred vozidlom. Použitý HC-SR04 ultrazvukový senzor funguje ako vysielateľ, prijímač aj riadiaci obvod – obrázok 9 ilustruje jeho funkciu.



Obr. 9: Diagram fungovania ultrazvukového senzora

Princíp merania vzdialenosti je nasledovný:

1. Arduino pošle signál vysokej úrovne na Trig pin ultrazvukového senzora.
2. Modul pošle osem 40 kHz signálov a nastaví vysokú úroveň na pine Echo.
3. Spustí sa časovač, zatiaľ čo senzor očakáva návratový signál.
4. Ak zaznamená pulz, zastaví časovač a zníži hodnotu Echo pinu.

Na základe známej rýchlosti zvuku ($343 \text{ m}\cdot\text{s}^{-1}$) a času, za ktorý sa signál vrátil, je možné vypočítať vzdialenosť vozidla a objektu pred ním. Rovnica na výpočet vzdialenosti je nasledovná:

$$\text{vzdialenosť} = \frac{\text{nameraný čas}}{2} \cdot \text{rýchlosť zvuku}$$

Ultrazvukový modul je riadený pomocou dvoch pinov mikrokontroléra Arduino. Jeden pin, nastavený na mód OUTPUT, sa používa na zaslanie vysokej hodnoty na modul. Druhý pin, nastavený na mód INPUT, je určený na získanie signálu zo senzora; na tento pin bola pripojená rutinu prerušenia.

```
1 void IRAM_ATTR echoISR() {  
2   if (digitalRead(ECHO_PIN)) {  
3     echoStart = micros();  
4   } else {  
5     echoEnd = micros();  
6     echoDone = true;  
7   }  
8 }
```

Výpis kódu 5: Obsluha prerušenia ultrazvukového senzora

Keďže je tento výpočet náročnejší, merania prebiehajú v 50 ms intervaloch. Samotný výpočet sa vykonáva v rámci danej iterácie hlavného cyklu.

```
1 if (echoDone) {  
2   uint32_t duration = echoEnd - echoStart;  
3   distance = duration / 58;  
4   echoDone = false;  
5 }
```

Výpis kódu 6: Výpočet vzdialenosti z merania ultrazvukového senzora

Z dôvodu zjednodušenia výpočtu sa používa aproximácia skutočnej vzdialenosti v centimetroch. Ak sa meraný predmet nachádza vo vzdialenosti do 10 cm, vozidlo zastaví [18].

6.2.2 Ultralytics YOLOv8

V našej implementácii používame YOLOv8 od spoločnosti Ultralytics – ako bolo popísané v sekcii 5.5.1. Pre programovací jazyk Python existuje jednoduché rozhranie, ktoré prácu s modelom zjednodušuje. Inštalácia prebehla pomocou správcu balíkov pip.

Nasledovný kód nám vytvorí inštanciu predtrénovaného modelu `yolov8s.pt` a zadefinuje funkciu, pomocou ktorej vieme detegovať objekty.

```

1 model = YOLO("yolov8s.pt")
2
3 def detect(frame):
4     return model(frame, verbose=False)[0]

```

Výpis kódu 7: Príprava CNN na detekciu objektov

Funkcia `detect` vracia zoznam (list) objektov `Result`, napriek tomu, že vstupom je iba jedna snímka. Nultý prvok zoznamu obsahuje všetky detekcie pre daný obrázok.

6.3 Interpretácia dát

Správna interpretácia výstupných dát z jednotlivých modulov je dôležitá pre spoluprácu senzorov a pohybových systémov vozidla. V autonómnych autách túto úlohu plní modul plánovania a rozhodovania, ktorý spája subsystémy vnímania a samotného konania.

Ak bola kamera úspešne pripojená a je možné načítať jej obraz, prechádzame k jeho samotnému spracovaniu.

Implementáciu sme začali detekciou prekážok pomocou CNN. Na zvýšenie výpočtovej efektivity programu sa táto detekcia spúšťa každých 6 snímok. Keďže model YOLOv8 dokáže detegovať až 80 rôznych tried objektov – vrátane tých, ktoré sa na vozovke bežne nevyskytujú – vybrala sa iba ich špecifická podmnožina, čím sa predchádza falošným pozitívam. Do tohto výberu boli zahrnutí ľudia, dopravné prostriedky a zvieratá.

V prípade, že model YOLO identifikuje objekt z definovaného zoznamu prekážok, systém následne vyhodnotí podmienky na zastavenie vozidla. Rozhodovací proces sa opiera o tri hlavné podmienky, ktoré boli stanovené na základe testovania a optimalizácie systému:

1. **Trajektória** Prvým krokom je určenie pozície prekážky vzhľadom na jazdnú dráhu vozidla. Podmienka je splnená v prípade, že sa objekt nachádza v rámci definovanej hranice. Hranice sa zväčšujú podľa blízkosti objektu k vozidlu.
2. **Blízkosť objektu** V druhom kroku určujeme vzdialenosť k objektu, ktorú aproximujeme na základe spodnej vertikálnej súradnice detegovanej prekážky.
3. **Spôľahlivosť** Záverečná podmienka je overenie miery konfidencie modelu. Týmto sa minimalizujú falošné pozitíva a nežiadúce zastavenie vozidla.

Až po súčasnom splnení všetkých troch podmienok sa aktivuje zastavenie vozidla.

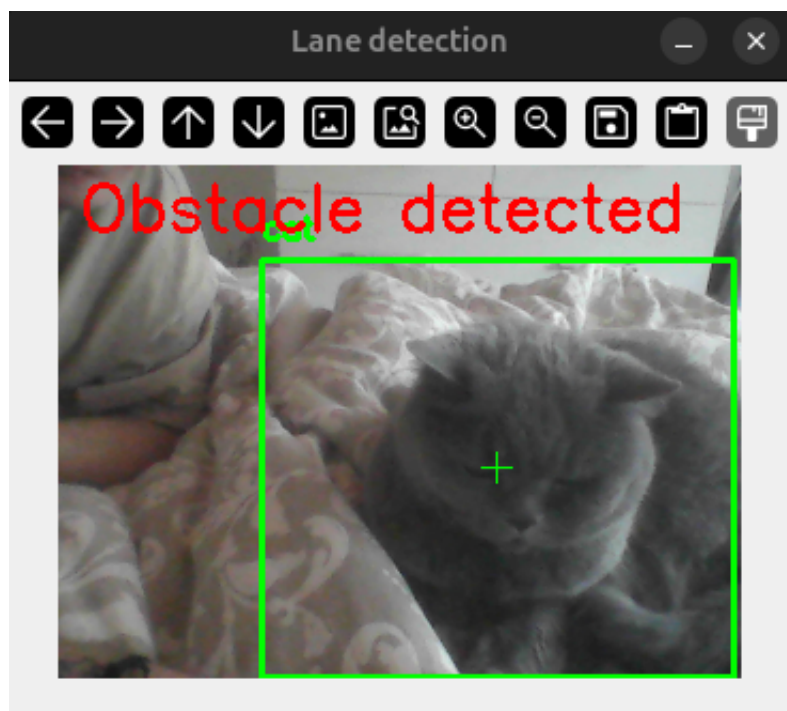
```

1 def decide_stop(det, alpha = 0.5):
2     for box in det.bboxes:
3         cls = int(box.cls[0])
4         name = model.names[cls]
5
6         if name in obstructions:
7             x1, y1, x2, y2 = map(int, box.xyxy[0])
8
9             conf = float(box.conf[0])
10            margin = int(y2 * alpha)
11
12            in_front = x1 < (CENTER + margin) and x2 > (CENTER - margin)
13            close = y2 > CLOSE_THRESHOLD
14            reliable = conf > CONF_THRESHOLD
15
16            if in_front and close and reliable:
17                return True, box
18    return False, None

```

Výpis kódu 8: Rozhodovanie zastavenia pri detekcii pomocou CNN

Posledný krok je zaznačiť detekciu na snímke – viď obrázok 10.



Obr. 10: Detekcia prekážky pomocou CNN.

V alternatívnom scenári, kde nebola nedetegovaná prekážka pomocou CNN, postupujeme s určením jazdného pruhu.

Po identifikácii kontúr jazdného pruhu je ďalším krokom určenie stredovej línie tej najvýraznejšej z nich. Vyberieme si kontúru s najväčšou plochou. Pomocou funkcie `moments` z knižnice OpenCV získame momenty – vážené priemery intezity (jasu) – jednotlivých pixelov. Tieto údaje sú dôležité pri výpočte geometrického stredu kontúry [23].

Súradnice stredu (c_x, c_y) vypočítame pomocou nasledujúcich vzorcov:

- **X-ová súradnica** $c_x = \frac{m_{10}}{m_{00}}$
- **Y-ová súradnica** $c_y = \frac{m_{01}}{m_{00}}$

Kde m_{10} a m_{01} predstavujú horizontálny a vertikálny moment a m_{00} zodpovedá ploche danej kontúry.

```

1 def find_center(contours):
2     if contours:
3         largest = max(contours, key=cv.contourArea)
4         M = cv.moments(largest)
5         if M["m00"] != 0:
6             cx = int(M["m10"] / M["m00"])
7             cy = int(M["m01"] / M["m00"])
8
9             return cx, cy
10    return None, None

```

Výpis kódu 9: Výpočet geometrického stredu kontúry

Po zistení geometrického stredu jazdného pruhu bude vypočítaná samotná hodnota riadenia. Naša implementácia normalizuje hodnoty na rozsah hodnôt $< 1, 511 >$, čím zabezpečí plynulé vstupy pre riadiaci systém vozidla.

- **Hodnoty $< 1, 255 >$:** Otáčanie doľava.
- **Hodnoty $< 257, 511 >$:** Otáčanie doprava.
- **Hodnota 256:** Priama jazda.
- **Hodnota 0:** Rezervovaná pre zastavenie.

Tento normalizovaný postup priraduje odchýlku zo stredu priamo k hodnote riadenia. Aplikáciou nelineárnej mocninovej funkcie transformujeme výstupný rozsah tak, aby sme v okolí stredovej osi dosiahli vyššiu stabilitu a presnosť riadenia.

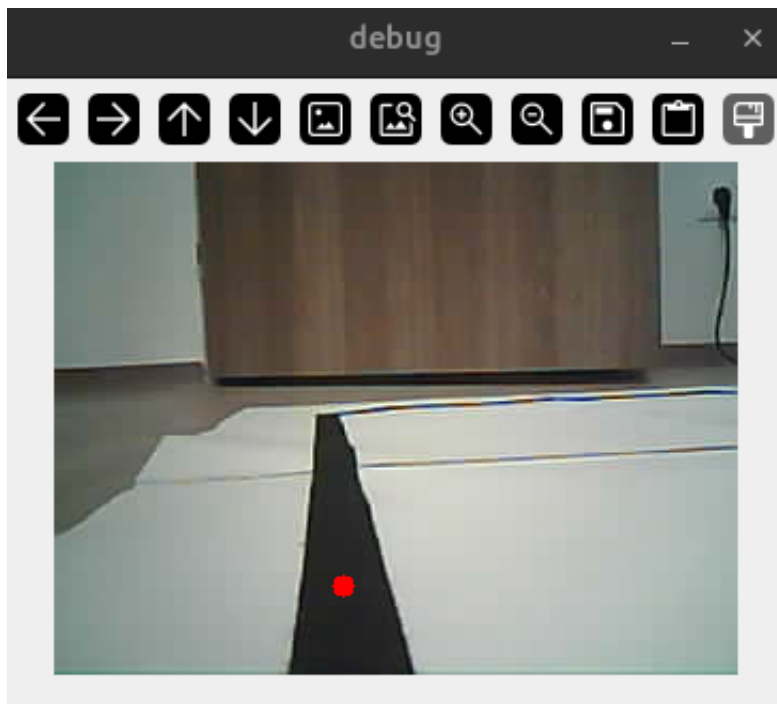
```

1 def compute_steering(cx, center=160, alpha=1.6):
2     error = (cx - center) / center
3     error = pow(abs(error), alpha) * (1 if error >= 0 else -1)
4
5     return int(256 + error * 255)

```

Výpis kódu 10: Výpočet hodnoty riadenia

Pomocou zistených hodnôt sa zaznačí stred jazdného pruhu a vypíše hodnota otáčania, ako je možné vidieť na obrázku 11.



Obr. 11: Hľadanie stredu jazdného pruhu.

Riadiaci program na platforme Arduino najskôr kontroluje údaje z ultrazvukového modulu. Ak senzor deteguje prekážku vo vzdialenosti do desiatich centimetrov, vozidlo okamžite zastaví. Tento ultrazvukový snímač má v celom systéme najvyššiu prioritu.

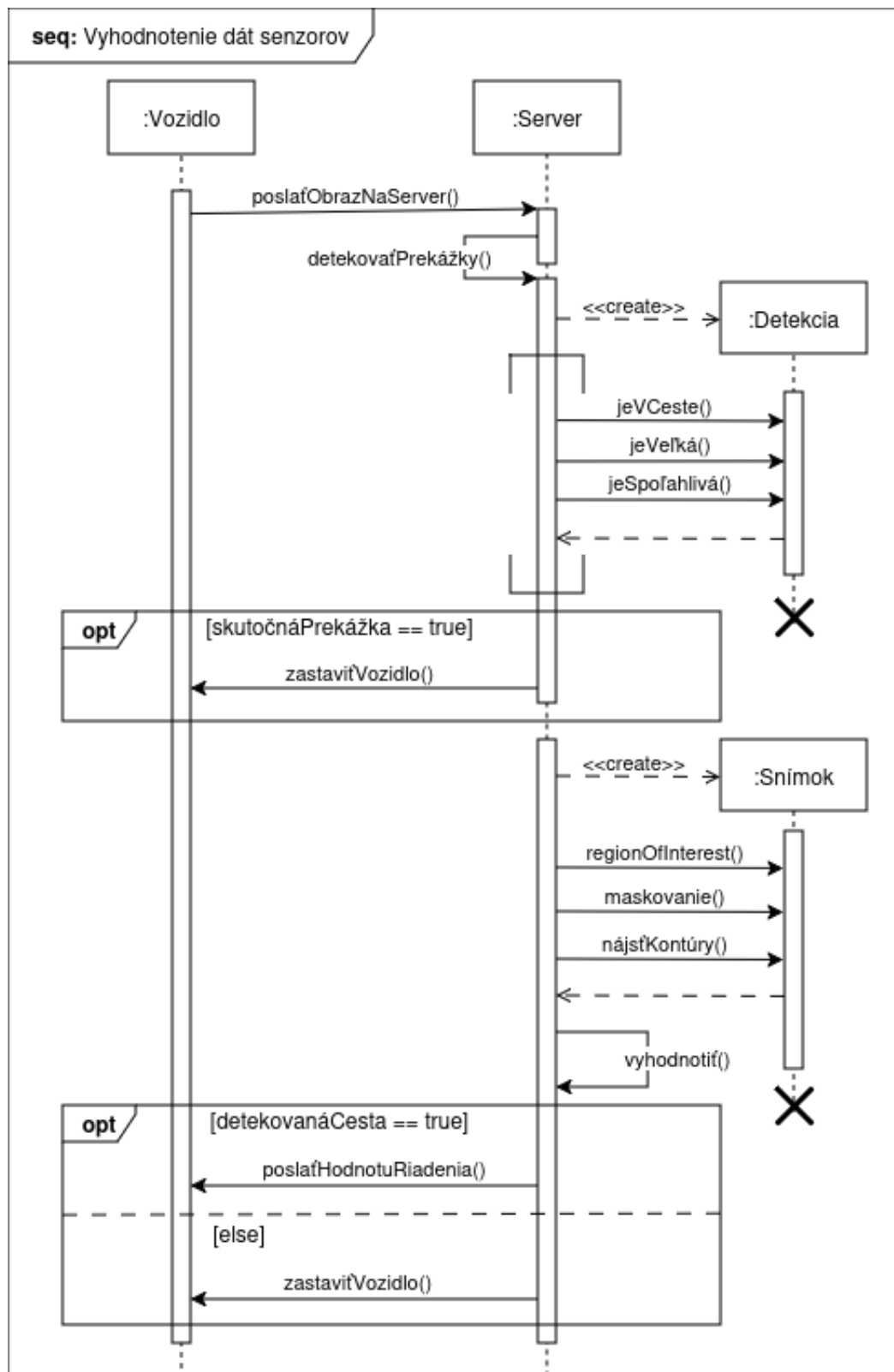
```

1 if (distance < 10 || steer < -255 || steer > 255) {
2     stopWheels();
3 } else {
4     driveSteering(steer);
5 }

```

Výpis kódu 11: Rozhodovanie na vozidle

Celkovú postupnosť krokov v tejto podkapitole opisuje nasledovný sekvenčný diagram.

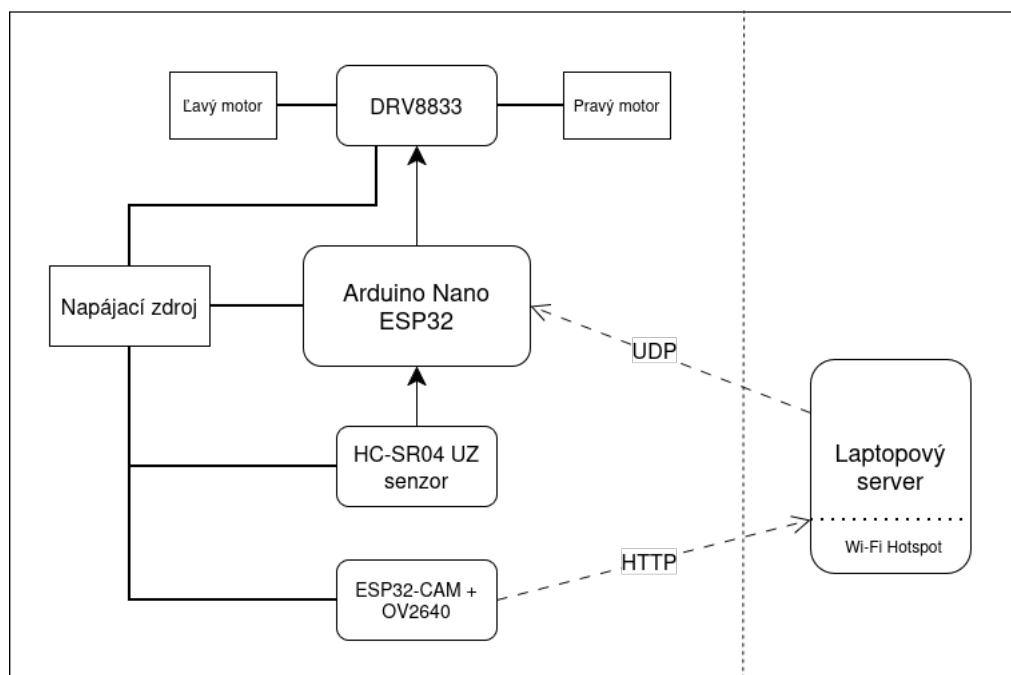


Obr. 12: Sekvenčný diagram vyhodnotenia dát zo senzorov.

6.4 Návrh komunikácie komponentov

Komunikácia komponentov predstavuje dôležitý aspekt projektu, pretože je nutné aby si jednotlivé časti mohli synchronne vymieňať a spracovať dáta.

Dôsledkom nízkeho výpočtového výkonu mikrokontrolérov Arduino sme použili v tejto implementácii model klient-server. V danej štruktúre vystupujú dva typy entít; nadradení klienti iniciujú požiadavky na pasívny server, ktorý posiela odpovede klientom.



Obr. 13: Diagram komunikácie komponentov

6.4.1 Komunikácia kamery a serveru

Server je v našom projekte laptop, ktorý taktiež spúšťa Wi-Fi sieť na komunikáciu s komponentmi – viď obrázok 13.

Všetky funkcie webového servera boli implementované v jazyku Python. Bezdrôtový prístupový bod bol vytvorený prostredníctvom nástroja NetworkManager, ktorý slúži na konfiguráciu sieťových rozhraní v operačnom systéme Linux. S jeho pomocou bola nakonfigurovaná sieťová karta laptopu do režimu Wi-Fi hotspotu. Na spustenie tohto systémového nástroja priamo z Pythonu bola využitá knižnica subprocess.

```

1 SSID = "LAPTOP_AP"
2 PASSWORD = "12345678"
3
4 def startHotspot():
5     print("Starting Hotspot")
6     subprocess.run([
7         "nmcli",
8         "device",
9         "wifi",
10        "hotspot",
11        "ssid", SSID,
12        "password", PASSWORD
13    ], check=True)

```

Výpis kódu 12: Zapnutie Wi-Fi hotspotu

Pred spustením cyklu spracovania obrazu bolo nutné počkať na pripojenie kamerového modulu k Wi-Fi sieti. Dostupnosť bola zisťovaná cez protokol ICMP vysielaním ping paketov v 1-sekundových intervaloch. Výstup s hodnotou 0 potvrdzuje úspešné pripojenie kamery.

```

1 def ping_cam():
2     while True:
3         response = os.system(f"ping -c 1 {CAM_IP} > /dev/null 2>&1")
4         if response == 0:
5             print("ESP32-CAM online")
6             return
7         print(".", end="")
8         time.sleep(1)

```

Výpis kódu 13: Pingovanie IP adresy kamerového modulu

Modul ESP32-CAM pracuje v režime klienta, ktorý prostredníctvom Wi-Fi siete odosiela obrazové dáta na server. Po nadviazaní spojenia začne cez protokol HTTP na porte 81 vysielat video stream z kamery OV2640. Snímky sú prenášané v rozlíšení QVGA (320 × 240 pixelov) vo formáte JPEG. Nová snímka sa odosiela každých 30 ms, vďaka čomu dosahuje výsledný stream snímkovú frekvenciu približne 30 FPS.

```

1 void startCameraServer(){
2     httpd_config_t config = HTTPD_DEFAULT_CONFIG();
3     config.server_port = 81; // port streamu
4
5     if(httpd_start(&stream_httpd, &config) == ESP_OK){
6         httpd_uri_t stream_uri = {
7             .uri          = "/stream",
8             .method       = HTTP_GET,
9             .handler       = stream_handler,
10            .user_ctx      = NULL
11        };
12        httpd_register_uri_handler(stream_httpd, &stream_uri);
13    }
14 }

```

Výpis kódu 14: Spustenie HTTP stream na ESP32-CAM

6.4.2 Komunikácia serveru a mikrokontroléra

Vďaka statickej IP adrese bolo možné určiť, na ktorej URL má server očakávať obrázky. Na ich načítanie slúži funkcia VideoCapture z OpenCV, ktorá podporuje spracovanie obrazu z URL. Po analýze dát server odošle inštrukcie pre Arduino Nano. Odpoveď sa posielala cez UDP socket na port 5005 mikrokontroléra, čo je optimálne pre aplikácie v reálnom čase. Komunikáciu zabezpečuje knižnica socket v Pythone a posielené dáta majú veľkosť len 2 bajty, čo je najmenší nutný buffer na uloženie informácií o riadení.

```

1 CAM_IP = "10.42.0.128"
2 CAM_URL = f"http://{CAM_IP}:81/stream"
3
4 UDP_IP = "10.42.0.111"
5 UDP_PORT = 5005
6
7 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8
9 def sendUDP(direction: float):
10     sock.sendto(struct.pack('<H', direction), (UDP_IP, UDP_PORT))

```

Výpis kódu 15: Posielanie UDP packetov

6.4.3 Komunikácia mikrokontroléra s perifériami

Arduino prijme UDP packet a súčasne získa nameranú vzdialenosť z ultrazvukového senzora. Keďže Echo pin na senzore, z ktorého získavame signál, má výstupné napätie 5 V, je potrebné ho znížiť na 3,3 V, čo je operačné napätie všetkých GPIO pinov mikrokontroléra.

Toto bolo dosiahnuté pomocou deliča napätia. Výstupné napätia deliča bolo vypočítané pomocou rovnice $V_{OUT} = \frac{V_{IN} \cdot R_2}{R_1 + R_2}$, kde R_1 a R_2 sú odpory dvoch rezistorov zapojených sériovo, V_{IN} je vstupné napätie a V_{OUT} výstupné napätie. Bolo určené, že je potrebné zapojiť $10\text{ k}\Omega$ a $20\text{ k}\Omega$ odpor v sérii.

Ultrazvukový snímač sa spúšťa v intervaloch 50ms, z dôvodu efektívneho využitia výpočtového výkonu. Jednotlivé spustenia sú vo forme vyvolania prerušenia.

Každému z motorčekov sú priradené dva výstupné piny – pinA a pinB. Všetky štyri piny sú pripojené na osobitný PWM kanál. Pulse Width Modulation (PWM) nám dovoľuje presnú kontrolu nad rýchlosťou motorčekov. Kanály boli nakonfigurované pomocou funkcie `ledcSetup`. V danej funkcii bola nastavená frekvencia menenia (PWM_FREQ) na 20kHz a rozlíšenie (PWM_RES) na 8. Takéto rozlíšenie dovoľuje 256 (2^8) rôznych stupňov rýchlosti. Následne boli funkciou `ledcAttachPin` priradené piny ku kanálom.

Signály z pinov sú privedené na DRV8833, ktorý fyzicky riadi dva DC motory. Na tento účel bola využitá funkcia `ledcWrite` z knižnice Arduino určenej pre mikrokontroléry ESP32. V prípade, že má motor sa otáčať vpred, na pin A sa zapíše hodnota rýchlosti a pin B sa nastaví na nulu. V opačnom scenári sa stavy pinov vymenia. Ak je potrebné motor zastaviť, na oba piny sa zapíše hodnota 0, pričom rovnaký efekt by sa dosiahol aj zápisom vysokých hodnôt na oba piny súčasne.

```

1 void loop() {
2   while (udp.parsePacket() >= 2) {
3     udp.read(incoming, 2);
4
5     uint16_t raw = incoming[0] | (incoming[1] << 8);
6     steer = (int)raw - 256;
7   }
8
9   static unsigned long lastTrig = 0;
10  if (millis() - lastTrig > 50) {
11    lastTrig = millis();
12    echoDone = false;
13    triggerUltrasonic();
14  }
15
16  if (echoDone) {
17    uint32_t duration = echoEnd - echoStart;
18    distance = duration / 58;
19    echoDone = false;
20  }
21 }

```

Výpis kódu 16: Vnútorý cyklus Arduina

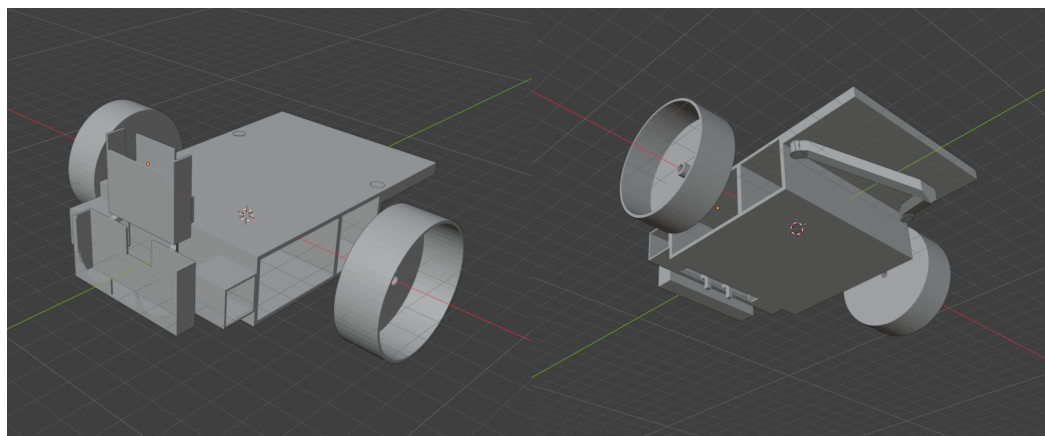
6.5 Návrh rámu a rozloženie komponentov

Správny návrh rámu a rozloženia komponentov na ňom je dôležitý pre správne fungovanie celého systému. V našej implementácii sme sa zamerali na minimalizovanie hmotnosti vozidla a maximalizovanie kompaktnosti komponentov, a tým ich zohranie.

Na návrh rámu sme použili softvér Blender – populárny, všeobecne používaný program na dizajn, modelovanie a animáciu, ktorý umožňuje exportovať do formátu .stl, ideálneho pre 3D tlač. Po odmeraní všetkých komponentov, sme začali pracovať na prototype rámu.

V procese vývoja bolo vyskúšaných viacero návrhov, v rámci ktorých bolo testované rôzne usporiadania a orientácie jednotlivých súčiastok. Napokon bol zvolený dizajn s dvoma kolesami vpredú a vymeniteľnou zadnou vidlicou, ktorú je možné v prípade opotrebovania alebo zmeny povrchu jednoducho nahradiť.

Najťažšie komponenty – konkrétne batériu a menič napätia – sme umiestnili na spodnú stranu vozidla s cieľom znížiť ťažisko a zvýšiť celkovú stabilitu. Konštrukciu sme zároveň doplnili o držiak na kameru a ultrazvukový senzor na zabezpečenie stabilizácie obrazu.



Obr. 14: 3D model vozidla

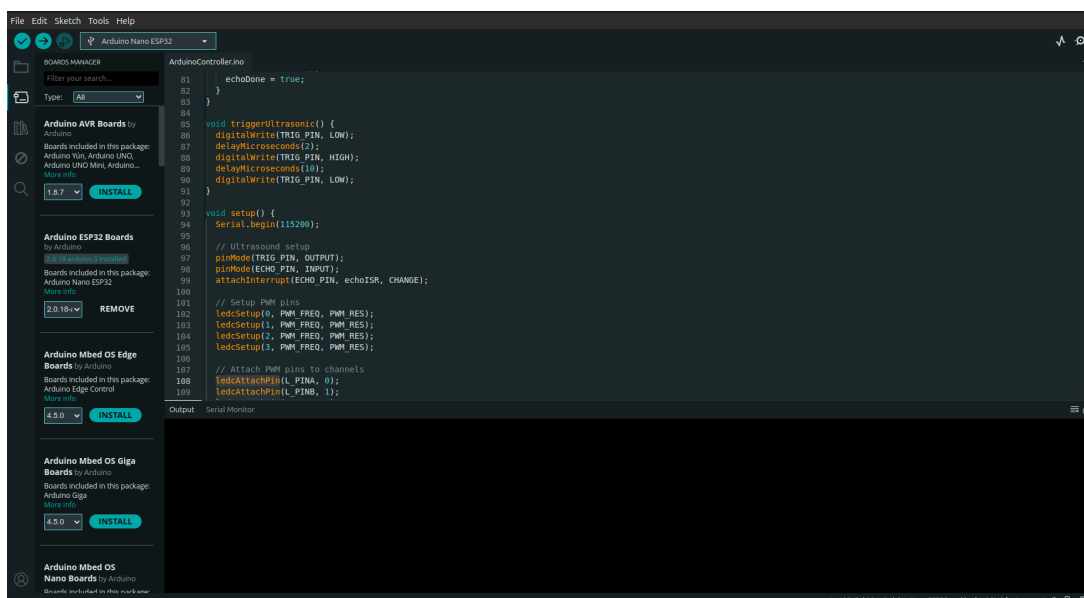
Model bol vytlačený na tlačiarni Bambulab A1 mini, použitím filamentu kyseliny polymliečnej (PLA). Prvotne boli kolesá vytlačené z polyetyléntereftelátu (PET) a doplnené o protišmykový polep, no tento povrch nedosahoval požadovaný stupeň trakcie.

Zakúpili sme dve kolesá s priemerom 43mm, gumeným povrchom a oskou určenou pre náš typ motorčekov.

6.6 Montovanie vozidla a nahratie softvéru

V tejto podkapitole si priblížime fyzické spojenie hardvéru a softvéru. Priebeh procesu nebol priamočiary a vykazoval časové aj logické odchýlky od predpokladaného modelu. Mnohé spomenuté komponenty boli testované samostatne a skutočný program prešiel mnohými iteráciami, predtým než sme našli tú ideálnu. Pre potreby tejto práce však uvádzame optimálnu sekvenciu krokov, ktoré predstavujú logicky správny postup.

Začali sme nahratím kódu na Arduino Nano ESP32. Tento mikrokontrolér má podporu nahrávania kódu pomocou konektoru USB-C. Integrované vývojové prostredie Arduino IDE (obrázok 15) automaticky detegovalo pripojený mikrokontrolér. Pomocou tlačidla Upload v tomto prostredí bol úspešne skompilovaný a nahratý kód do pamäte Arduina.



Obr. 15: Integrované vývojové prostredie Arduino

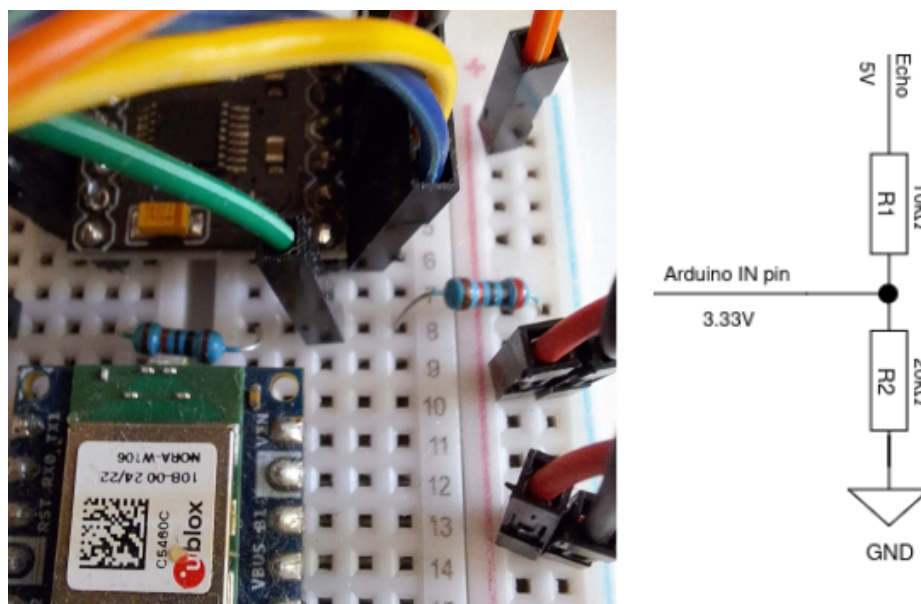
V ďalšom kroku bol nahratý kód na ESP32-CAM. Na rozdiel od Arduina, ESP32-CAM nemá integrovaný USB konektor. Na tento účel bol k nemu pripojený programovací shield určený pre túto dosku. Tento modul funguje ako adaptér USB na sériové rozhranie a disponuje starším Micro-USB portom. Podobne ako v minulom kroku, Arduino IDE rozpoznalo vývojovú dosku a kód bol pomocou tohto prostredia skompilovaný a nahratý. Týmto krokom je nahrávanie softvéru pre hardvérovú časť vozidla hotové.

Na prepojenie súčiastok sme použili nepájivé pole a farebne odlíšené vodiče. Breadboard bol pripevnený na rám vozidla pomocou lepiacej vrstvy na jeho spodnej strane.

Vďaka headerom bolo Arduino Nano ESP32 a driver motorov DRV8833 zapojené priamo do breadboardu.

Podľa konfigurácie boli prepojené štyri výstupné piny Arduina pre ľavý a pravý motor s IN pinmi driverom motorov. K motorom boli prispájkované káble a pripevnené kolesá. Celá sústava pre obe strany bola následne zasunutá do vyhradených držiakov. Konce týchto káblov sme zapojili do OUT pinov drivera motorov.

V ďalšom kroku bol zapojený ultrazvukový senzor. Trig pin senzora bol spojený s určeným výstupným pinom na Arduino pomocou vodiča. Na Echo pin senzora bol pripojený $10\text{ k}\Omega$ rezistor a $20\text{ k}\Omega$ rezistor bol zapojený cez spoločné uzemnenie. Výstupné napätie $3,333\text{ V}$ – prijateľné pre Arduino – bolo vyvedené z vodiča pripojeného medzi týmito dvoma odpormi. Koniec tohto vodiča bol zapojený do vstupného pinu na Arduino – viď obrázok 16.



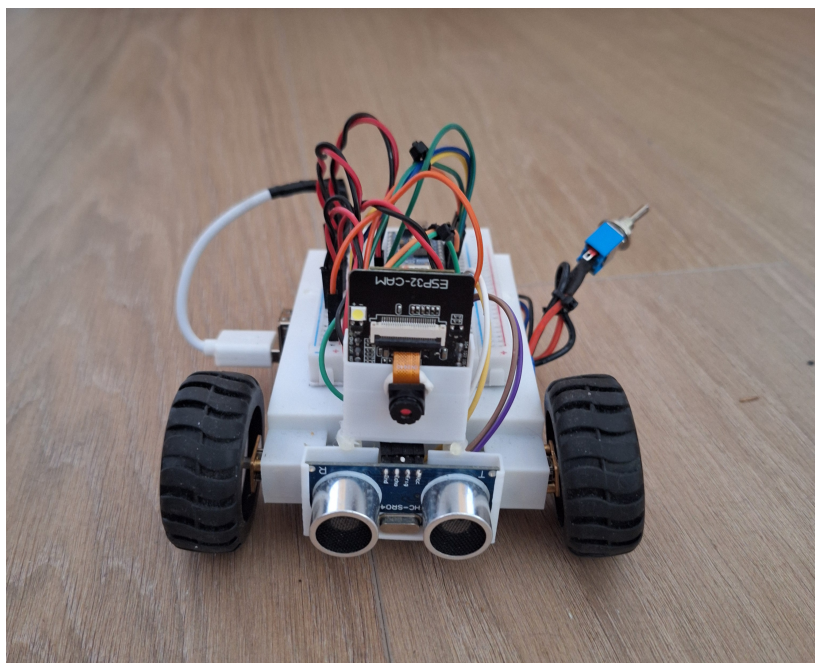
Obr. 16: Delič napätia

Ultrazvukový senzor a kamerový modul boli vložené do držiaka. Následne bol držiak nasadený na dva podporné body nachádzajúce sa na ráme vozidla. Pomocou sťahovacích pásov bol držiak pripevnený.

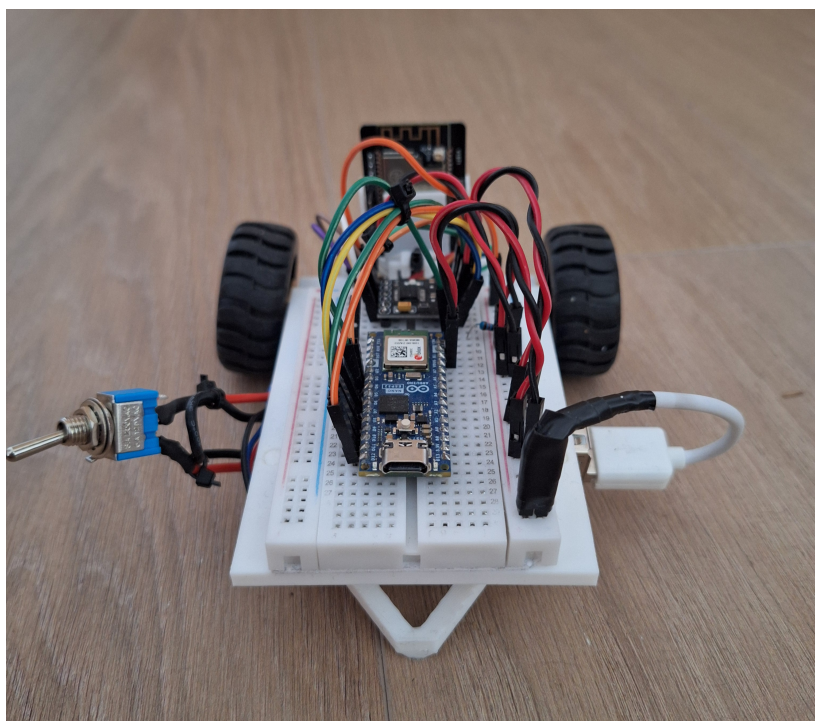
V ďalšom kroku bola zapojená batéria. Na vodič kladného pólu batérie bol pripevnený vypínač a následne boli oba káble zapojené do buck meniča. K výstupnému portu meniča bol pripojený modifikovaný USB kábel, ktorého koncový konektor bol odstránený. K jeho červenému a čiernemu vodiču boli pripevnené kontakty – kladný a záporný. Tento upravený konektor bol potom zapojený do napájacej koľajnice breadboardu.

Mikrokontrolér, kamerový modul, motor driver a ultrazvukový senzor boli zapojené na napájaciu koľajnicu pomocou čiernych a červených káblikov.

Na záver bola do rámu vložená podporná vidlica, čím bola montáž vozidla skompletizovaná – viď obrázok 17 a 18.



Obr. 17: Skompletizované vozidlo (predok)



Obr. 18: Skompletizované vozidlo (zadok)

7 Metodika testovania

V tejto kapitole sú popísané použité techniky a proces testovania navrhnutého systému. Testovanie je nevyhnutnou súčasťou realizácie každého projektu, pretože umožňuje overiť funkčnosť návrhu v praxi a posúdiť mieru naplnenia stanovených cieľov v reálnych podmienkach. Výsledky tejto časti zároveň slúžia na porovnanie dosiahnutých riešení s inými prácami, ktoré využívajú odlišnú architektúru.

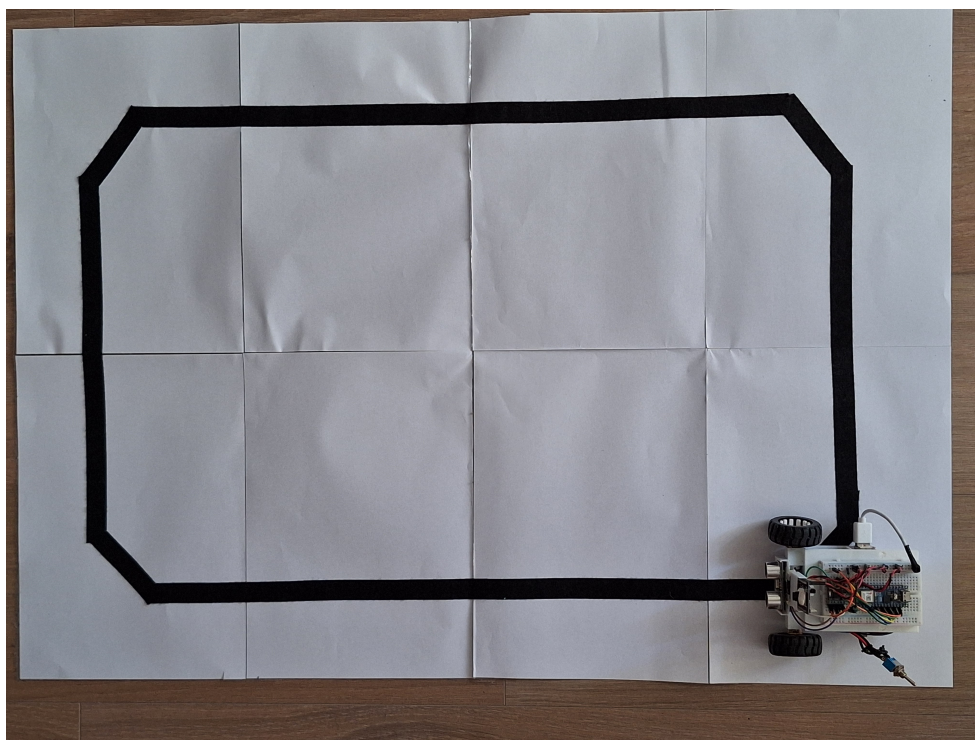
V rámci experimentov sme sa zamerali na overenie dvoch hlavných funkcií systému:

- **Autonómne riadenie** Cieľom bolo otestovať schopnosť vozidla sledovať vopred určenú trasu. Testovacia dráha bola navrhnutá tak, aby obsahovala rovné úseky aj zákruty. Úspešnosť systému sme hodnotili podľa času dokončenia a schopnosti vozidla udržať sa na vozovke bez vybočenia.
- **Detekcia prekážky** Keďže vývoj tohto systému bol hlavným cieľom práce, väčšina testov je zameraná práve na túto oblasť. Systém využíva dva typy senzorov – kamerový modul s konvolučnou neurónovou sieťou a ultrazvukový senzor. Testy sú navrhnuté tak, aby preverili spoľahlivosť oboch týchto systémov.

7.1 Autonómne riadenie

Autonómne riadenie chápeme, ako schopnosť vozidla samostatne vykonávať jazdné úkony bez zásahu ľudského operátora. Pre overenie tejto schopnosti musí riadiaci systém vozidla zvládať jazdu nielen na priamych úsekoch, ale aj v zákrutách. Z tohto dôvodu bolo nevyhnutné navrhnuť testovaciu dráhu, ktorá zahŕňa obidva prvky. Grafický návrh trate bol pôvodne vypracovaný v prostredí editora obrázkov.

Po grafickom návrhu nasledovala fyzická realizácia trate. Na jej vytvorenie sme použili osem papierov formátu A4 (celkovou plochou zodpovedajúcich formátu A1), ktoré boli vzájomne prepojené do súvislého celku. Samotná dráha bola následne vytvorená pomocou čiernej matnej lepiacej pásky, pričom sme ako presnú predlohu využili digitálny dizajn (obrázok 19).



Obr. 19: Testovacia trať

Proces testovania spočíva v sledovaní dvoch hlavných parametrov: schopnosti vozidla prejsť celú dráhu bez jej opustenia a času, za ktorý okruh absolvuje. Meranie prebiehalo v interiéroých podmienkach pri ambientnom dennom svetle. Čas prejazdu vozidla cez stanovenú dráhu sme merali od momentu prvotného pohybu až po opätovné prekročenie štartovacej čiary (dokončenie okruhu).

Pri testovaní sme sledovali aj chybovosť, konkrétne nedokončenie dráhy. Tento jav sme definovali v dvoch prípadoch:

- ak sa vozidlo permanentne zastavilo v dôsledku straty detekcie vodiacej čiary,
- ak vozidlo opustilo jazdný pruh z dôvodu chybnej detekcie objektov mimo dráhy.

V rámci tohto testu detekciu prekážok sme nesledovali. Prípadné zastavenie sa preto nepovažovalo za chybu a test bol zopakovaný.

Test sme vykonali celkovo desaťkrát. Všetky namerané výsledky zaznamenali do tabuľky, aby sme mohli následne vyhodnotiť úspešnosť a stabilitu riadenia.

7.2 Detekcia prekážky

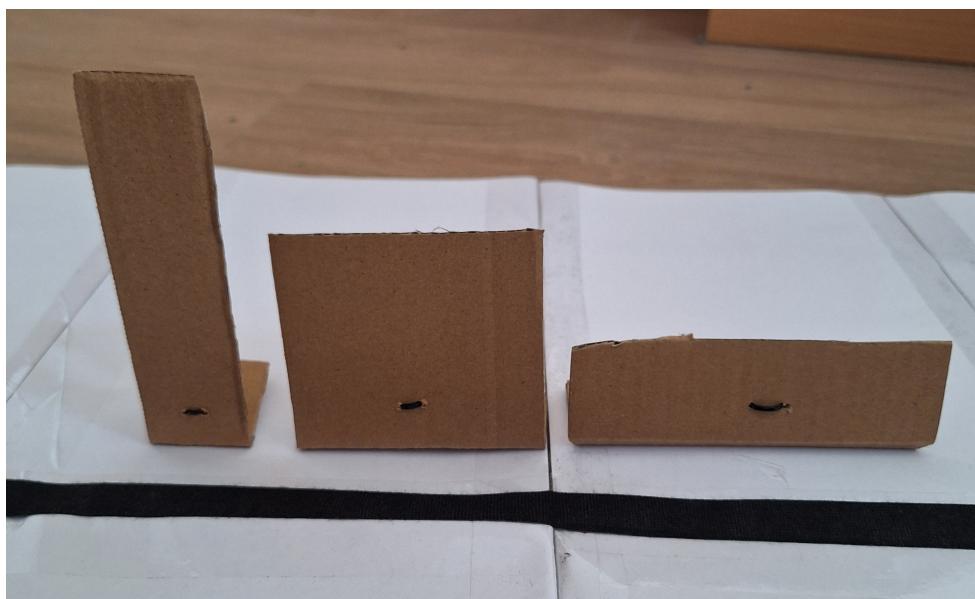
Významnou vlastnosťou autonómneho riadenia je schopnosť vozidla detegovať prekážky a vhodne na ne reagovať. Väčšina systémov využíva určitú mieru redundancie, čo

znamená, že vozidlo disponuje záložným riešením pre prípad zlyhania niektorého z podsystémov.

Z tohto dôvodu sme testovanie rozdelili do dvoch častí:

- Testovanie ultrazvukového modulu.
- Testovanie konvolučnej neurónovej siete.

Na obrázku 20 sú znázornené objekty využité pri testovaní ultrazvukového senzoru.



Obr. 20: Prekážky na testovanie ultrazvukového modulu.

Konkrétne ide o tieto typy prekážok:

- **Štíhla prekážka** (4x14 cm): Slúži na zobrazenie prekážok s vertikálnou orientáciou, ako sú napríklad stĺpy alebo ľudia.
- **Stredne veľká prekážka** (9x9 cm): Predstavuje univerzálny typ objektu a reprezentuje rozmerovo priemerné prvky prostredia, napríklad časti stien či vozidiel.
- **Široká prekážka** (14x4 cm): Je určená na zobrazenie nízkych prekážok s horizontálnou orientáciou, kam môžeme zaradiť nízke oplotenie, menšie zvieratá alebo deti.

Proces testovania bol rozdelený do troch fáz podľa umiestnenia prekážky vzhľadom na jazdnú dráhu:

- **Fáza 1:** Prekážka je umiestnená priamo v strede vozovky.
- **Fáza 2:** Prekážka sa nachádza na okraji vozovky.
- **Fáza 3:** Prekážka je situovaná mimo vozovky.

V každej z týchto fáz sme sledovali a vyhodnocovali reakciu vozidla, konkrétne jeho schopnosť včas zastaviť, respektíve pokračovať v jazde v závislosti od polohy detegovaného objektu. Každý test sme opakovali päťkrát, čím sme získali celkovo štyridsaťpäť meraní.

Testovanie neurónovej siete prebiehalo s piatimi rôznymi objektmi: osobným automobilom, nákladným vozidlom, bicyklom, človekom a psom (obrázok 21). Na testovanie sme použili CNN YOLOv8 od spoločnosti Ultralytics. Sledovali sme nielen úspešnosť detekcie, ale aj to, či systém správne vyhodnotí polohu prekážky (priamo na vozovke alebo v jej blízkosti) a adekvátne na ňu zareaguje. Ultrazvukový senzor sme odpojili, aby jeho vstup neovplyvňoval výsledky neurónovej siete. Každú situáciu sme testovali päťkrát, čo pri kombinácii typu objektu a jeho polohy prinieslo spolu päťdesiat výsledkov.



Obr. 21: Prekážky na testovanie CNN.

Celkovo v tejto časti vyhodnocujeme deväťdesiatpäť testovacích scenárov. Na ich základe určíme efektívnosť navrhnutej implementácie a identifikujeme prípadné nedostatky systému detekcie prekážky.

8 Výsledky

V tejto kapitole sa zameriavame na analýzu dát získaných počas meraní opísaných v predchádzajúcej časti práce. Cieľom objektívneho zhodnotenia je získať relevantnú spätnú väzbu o úspešnosti navrhnutej implementácie. Namerané údaje zároveň slúžia ako podklad pre prípadné porovnanie s inými modelmi autonómnych vozidiel.

Proces vyhodnocovania sme rozdelili do troch častí, ktoré predstavujú jednotlivé systémy riadenia:

- Testovanie detekcie vozovky.
- Testovanie ultrazvukového senzora.
- Testovanie konvolučnej neurónovej siete.

8.1 Testovanie detekcie vozovky

Tabuľka 1: Detekcia jazdného pruhu

Poradie	Čas	Nedokončilo?
1	11.88 s	nie
2	12.35 s	nie
3	n/a	áno
4	13.13 s	nie
5	13.80 s	nie
6	12.48 s	nie
7	14.84 s	nie
8	12.89 s	nie
9	11.92 s	nie
10	11.38 s	nie
Priemer	12.74 s	90%

Tabuľka 1 zobrazuje výsledky testovania detekcie jazdného pruhu. Z desiatich testov bolo deväť úspešných, kde vozidlo neopustilo trať. V treťom teste sme zaznamenali zlyhanie, kedy vozidlo pri zatáčaní stratilo vizuálny kontakt s jazdným pruhom. Vozidlo nedokázalo znovu detegovať vozovku, a preto sme test označili ako neúspešný.

Priemerný čas dosiahol 12.74 s, pričom medián bol 12.48 sekundy. Rozmedzie výsledkov bolo 3.46 sekúnd. Najrýchlejší prejazd sme zaznamenali v desiatom pokuse (11.38 s). Najpomalší čas bol v siedmom pokuse (14.84 s). Tento údaj je významný,

keďže vozidlo v dôsledku dočasnej straty detekcie pruhu zastalo, čo predĺžilo prejazd o viac než dve sekundy oproti priemeru.

Vozidlo dokáže spoľahlivo nasledovať jazdný pruh na rovných úsekoch, no pri zabáčaní môže dôjsť k strate trajektórie. Vizualný šum z kamery spolu s vysokou rýchlosťou otáčania môže vyvolať nadmernú korekciu, alebo naopak nedostatočnú reakciu riadenia, čo vedie k opusteniu jazdného pruhu. V reálnych podmienkach využíva vozidlo ďalšie vizuálne pomôcky na predvídanie takýchto situácií, napríklad dopravné značenie.

8.2 Testovanie ultrazvukového senzora

Tabuľka 2: Ultrazvukový senzor (stred vozovky)

	Zastavilo?		
Poradie	Štíhla	Stredná	Široká
1	áno	áno	áno
2	áno	áno	áno
3	áno	áno	áno
4	áno	áno	áno
5	áno	áno	áno
Úspešnosť	100.00%	100.00%	100.00%

Tabuľka 2 hodnoty namerané pri umiestnení prekážky na stred vozovky. Ultrazvukový senzor nemal problém detegovať prekážku a zastaviť 10 cm pred ňou.

Tabuľka 3: Ultrazvukový senzor (mimo vozovky)

	Zastavilo?		
Poradie	Štíhla	Stredná	Široká
1	nie	nie	nie
2	nie	nie	nie
3	nie	nie	nie
4	nie	nie	nie
5	nie	nie	nie
Úspešnosť	100.00%	100.00%	100.00%

Podobne ako v predošlom teste, vozidlo nedetegovalo, že prekážka je v dráhe vozidla a správne vyhodnotilo, že nie je potrebné zastaviť vozidlo. Výsledky je možné vidieť v tabuľke 3.

Tabuľka 4: Ultrazvukový senzor (pri vozovke)

	Zastavilo?		
Poradie	Štíhla	Stredná	Široká
1	nie	nie	nie
2	nie	nie	nie
3	nie	nie	nie
4	nie	nie	nie
5	nie	nie	nie
Úspešnosť	0.00%	0.00%	0.00%

Tabuľka 4 zobrazuje kritické zlyhanie systému, ktoré priamo vyplýva z jeho technického návrhu. Ultrazvukové senzory sú primárne určené na detekciu objektov na krátke vzdialenosti pri nízkych rýchlostiach (napr. parkovacie manévry) a nie sú vhodné na všeobecnú detekciu prekážok. Pri reálnych automobiloch je tento nedostatok vyrovnaný rozmiestnením senzorov po celom obvode vozidla.

Použitý ultrazvukový modul HC-SR04 disponuje meracím uhlom 15° . Pri uplatnení sínusovej vety a stanovení brzdné vzdialenosti na 10 cm od senzora možno vypočítať, že efektívny priemer základne detekčného kužela dosahuje približne 2,6 cm. Pre pokrytie celej čelnej šírky vozidla (11 cm) by bolo nevyhnutné zvýšiť detekčnú vzdialenosť až na 42 cm. Takéto nastavenie by však spôsobovalo predčasné zastavenie vozidla a vnášalo do systému nežiaduce falošné detekcie.

8.3 Testovanie konvolučnej neurónovej siete

Tabuľka 5: Detekcia CNN – automobil

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Zastavilo?
1	áno	12 cm	áno
2	áno	13 cm	áno
3	áno	12 cm	áno
4	áno	11 cm	áno
5	áno	12 cm	nie
Úspešnosť	100.00%	12 cm	80.00%

Tabuľka 6: Detekcia CNN – nákladné auto

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Zastavilo?
1	áno	15 cm	áno
2	áno	16 cm	nie
3	áno	14 cm	áno
4	áno	15 cm	áno
5	áno	14 cm	áno
Úspešnosť	100.00%	14.8 cm	80.00%

Tabuľky 5 a 6 zobrazujú úspešnosť detekcie automobilu a nákladného auta. Ak sa objekt nachádza v strede vozovky, úspešnosť detekcie dosahuje 100%. Priemerná vzdialenosť zastavenia bola v tomto prípade 12 cm pri automobile a 14,8 cm pri nákladnom aute. Úspešnosť však klesla na 80%, ak sa prekážka nachádzala na okraji vozovky.

Tieto prekážky sú rozmerovo veľké, v dôsledku čoho často zaberajú celý záber kamery, prípadne sú len čiastočne viditeľné, ak sa nachádzajú na okraji vozovky. Detekcia je pomerne jednoduchá v situáciách, kedy sú objekty umiestnené v strede jazdného pruhu, pretože ich kľúčové črty sú dobre rozpoznateľné aj z väčšej vzdialenosti. Problém nastáva, ak sú prekážky viditeľné iba sčasti, čo spôsobuje pokles hodnoty istoty (confidence) CNN.

Tabuľka 7: Detekcia CNN – bicykel

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Zastavilo?
1	áno	10 cm	áno
2	áno	13 cm	áno
3	áno	11 cm	áno
4	áno	13 cm	áno
5	áno	12 cm	áno
Úspešnosť	100.00%	11.8 cm	100.00%

Tabuľka 8: Detekcia CNN – človek

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Zastavilo?
1	áno	15 cm	áno
2	áno	8 cm	áno
3	áno	12 cm	áno
4	áno	12 cm	áno
5	áno	9 cm	áno
Úspešnosť	100.00%	11.2 cm	100.00%

V tabuľkách 7 a 8 sú zobrazené výsledky testovania s prekážkami typu bicykel a človek. Úspešnosť zastavenia v oboch prípadoch dosiahla 100%. Priemerná vzdialenosť zastavenia bola 11,8 cm v prípade bicykla a 11,2 cm pri človeku, pričom rozptyl hodnôt bol vyšší ako v predchádzajúcich testoch.

Človek a bicykel predstavujú stredne veľké prekážky. Detekčný systém vykazoval o niečo nižšiu stabilitu pri rozpoznávaní týchto objektov v porovnaní s rozmerovo veľkými vozidlami, čo viedlo k väčšej variabilite vzdialenosti, v ktorej došlo k úspešnej detekcii. Na druhej strane, vďaka menším rozmerom týchto prekážok ich bolo možné spoľahlivo detegovať aj v prípade, ak sa nachádzali na okraji vozovky.

Tabuľka 9: Detekcia CNN – pes

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Zastavilo?
1	áno	7 cm	nie
2	áno	8 cm	áno
3	áno	7 cm	áno
4	áno	8 cm	áno
5	áno	8 cm	áno
Úspešnosť	100.00%	7.6 cm	80.00%

Tabuľka 9 zobrazuje výsledky detekcie pre triedu pes. Úspešnosť detekcie v strede vozovky dosahuje 100% s priemernou vzdialenosťou zastavenia 7,6 cm. Pri umiestnení prekážky na okraji vozovky úspešnosť klesla na 80%. V niektorých prípadoch CNN chybné klasifikovala prekážku typu pes ako inú triedu (napr. kôň alebo mačka).

Trieda pes predstavuje rozmerovo malý typ prekážky. V dôsledku nízkeho rozlíšenia kamery bolo rozpoznávanie tohto objektu náročné, čo sa prejavilo na nízkej priemernej

vzdialenosti zastavenia a nesprávnej klasifikácii objektu. Konvolučná sieť bola schopná prekážku detegovať a iniciovať zastavenie až pri jej značnom priblížení. Pri takejto kritickej vzdialenosti by už adekvátne zareagoval aj ultrazvukový senzor. V prípade zlyhania detekcie na okraji vozovky možno predpokladať, že v dôsledku oneskorenej reakcie algoritmu bola táto prekážka systémom úplne ignorovaná.

Z nameraných výsledkov je možné vyvodiť koreláciu medzi veľkosťou objektu a úspešnosťou jeho detekcie. Väčšie prekážky (osobné auto, nákladné auto) boli na detekciu jednoduchšie, no pri ich čiastočnom prekrytí alebo orezaní na okraji záberu bola úspešnosť rozpoznania nižšia. Naopak, menšie prekážky (človek, bicykel, pes) umožňujú detekciu v celom zornom poli kamery, avšak vzdialenosť potrebná na ich spoľahlivú detekciu je značne menšia.

Pre zvýšenie celkovej bezpečnosti a robustnosti vozidla v reálnych podmienkach sa preto ako nutné javí nasadenie dodatočných kamerových modulov na získanie 360 zorného poľa, implementácia senzorickej fúzie a taktiež rozšírenie tréningového datasetu pre umelú inteligenciu o špecifické hraničné scenáre.

Záver

V predloženej bakalárskej práci sa nám úspešne podarilo navrhnuť a implementovať funkčný model autonómneho vozidla s detekciou prekážok. Preskúmali sme pritom možnosti a obmedzenia platformy mikrokontrolérov Arduino a jej prepojenia so softvérom na počítačové videnie OpenCV. Pri samotnej implementácii riešenia sme skombinovali softvérové aj hardvérové prostriedky na detekciu prekážok.

V našej práci sme využívali rodinu mikrokontrolérov Arduino. Daný výrobca poskytuje nákladovo efektívne riešenia, avšak s nižším výpočtovým výkonom, preto sme preskúmali možnosť distribuovaného systému s architektúrou klient-server. Vďaka podpore bezdrôtovej komunikácie na doske Arduino Nano ESP32, ako aj na kamerovom module ESP32-CAM, sa nám podarilo vytvoriť jednoduchú sieť. Presunutím náročných výpočtov na server sme síce obetovali časť reakčnej rýchlosti, no získali sme vyššiu robustnosť systému. Na druhej strane je tento prístup limitovaný kvalitou pripojenia, a preto nie je vhodný pre bezpečnostno-kritické aplikácie.

Na autonómne riadenie sme použili algoritmus detekcie vozovky využívajúci funkcie z knižnice OpenCV. Tento prístup sa vyznačoval vysokou presnosťou na rovných úsekoch, avšak v zákrutách mohlo dôjsť k strate sledovanej dráhy. Tento nedostatok by sa dal optimalizovať pomocou kontextových pomôcok, ktoré by vozidlu signalizovali, aby spomalilo.

Na detekciu prekážok sme použili konvolučnú neurónovú sieť YOLOv8. Model S (small) sa ukázal ako ideálny pre aplikácie v reálnom čase s väčším výpočtovým výkonom, ktorým sme disponovali na laptope. Táto sieť bola schopná detekovať rôzne triedy objektov, čo bolo pre našu implementáciu dôležité, avšak pri malých a čiastočne prekrytých prekážkach narážala na problémy. Tento fakt vyplýva z obmedzeného rozlíšenia a úzkeho zorného poľa kamery. Integráciou viacerých kamier a dodatočným trénovaním neurónovej siete by však bolo možné tento problém úspešne eliminovať.

Sekundárnym systémom detekcie v našej implementácii bol ultrazvukový senzor. Jeho uplatnenie sa ukázalo ako veľmi presné a nákladovo efektívne v rámci jeho určeného využitia. Pri detekcii na krátke vzdialenosti vykazoval vysokú presnosť, avšak jeho efektívny detekčný kužeľ bol malý. V reálnej implementácii by preto bolo ideálne rozmiestniť viacero takýchto senzorov po celom obvode vozidla.

Naša implementácia sa zameriavala na otestovanie distribuovanej architektúry autonómneho riadenia. Hoci je znižovanie nákladov v oblasti autonómnych systémov výrazným trendom, nemožno opomenúť, že bezpečnosť je vždy prvoradá. Z tohto dôvodu sa domnievame, že takéto systémy nemajú v automobilovom priemysle priamu budúcnosť. Uplatnenie by však mohli nájsť (a čiastočne ho už nachádzajú) v oblasti

autonómneho doručovania tovaru, kde sú priemerná rýchlosť vozidiel a miera nebezpečenstva pre okolie pomerne nízke. Presunutím výpočtovej záťaže z edge na cloud computing by bolo možné výrazne znížiť výrobné náklady takejto doručovacej jednotky.

Na záver možno konštatovať, že predložená práca úspešne demonštruje uplatnenie platformy Arduino v autonómnej mobilite. Text detailne poukazuje na možnosti a nedostatky hardvéru i softvéru pri detekcii prekážok a vozovky. Práca zároveň prináša cenné poznatky o limitoch a praktickom využití distribuovanej architektúry, čím otvára nové otázky o budúcnosti a smerovaní autonómnej dopravy.

Literatúra

1. *2020 Autonomous Vehicle Technology Report* [online]. Wevolver, 2020. [cit. 2026-03-02]. Dostupné z: <https://www.wevolver.com/article/2020.autonomous.vehicle.technology.report>.
2. *J3016_202104: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles: Levels or categories of driving automation* [online]. Warrendale, PA, USA: SAE International, 2021 [cit. 2026-03-05]. Dostupné z: https://wiki.unece.org/download/attachments/128418539/SAE%20J3016_202104.pdf?api=v2.
3. PAREKH, D. et al. A Review on Autonomous Vehicles: Progress, Methods and Challenges. *Electronics* [online]. 2022 [cit. 2026-03-05]. ISSN 2079-9292. Dostupné z DOI: 10.3390/electronics11142162.
4. ANI KELKAR, K. H.; KELLNER, M. *Where to next? Insights from autonomous-vehicle experts* [online]. McKinsey & Company, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.mckinsey.com/features/mckinsey-center-for-future-mobility/our-insights/future-of-autonomous-vehicles-industry>.
5. MARKUS, F. *The Great Sensor War Behind Self-Driving Cars Just Got Way More Interesting* [online]. Motortrend, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.motortrend.com/news/latest-auto-lidar-radar-technology-ces-2026>.
6. SOARES, J. *NVIDIA Announces Alpamayo Family of Open-Source AI Models and Tools to Accelerate Safe, Reasoning-Based Autonomous Vehicle Development* [online]. Nvidia Newsroom, 2026. [cit. 2026-03-04]. Dostupné z: <https://nvidianews.nvidia.com/news/alpamayo-autonomous-vehicle-development>.
7. KOLLEWE, J. *Nvidia and UK Wealth Fund invest in British autonomous driving startup Oxa* [online]. The Guardian, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.theguardian.com/technology/2026/mar/04/nvidia-uk-wealth-fund-invest-british-autonomous-driving-startup-oxa>.
8. *Waymo Slims Sensors to Drive Down Robotaxi Costs* [online]. Autonomous Vehicles & AI, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.autonomous-vehicles-conference.com/news/waymo-slims-sensors-to-drive-down-robotaxi-costs>.
9. HAMZA, G.; ES-SADEK, M.; TAHER, Y. Artificial Intelligence In Self-Driving: Study of Advanced Current Applications [online]. 2023 [cit. 2026-03-08]. Dostupné z DOI: 10.31224/3209.

10. GRIGORESCU, S.; TRASNEA, B.; COCIAS, T.; MACESANU, G. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics* [online]. 2019, **37**, 362–386 [cit. 2026-03-08]. Dostupné z DOI: 10.1002/rob.21918.
11. BADUE, C. et al. Self-driving cars: A survey. *Expert Systems with Applications* [online]. 2021, **165**, 113816 [cit. 2026-03-08]. ISSN 0957-4174. Dostupné z DOI: <https://doi.org/10.1016/j.eswa.2020.113816>.
12. SINGH, K. *Tesla Launches Unsupervised Robotaxi Rides in Austin* [online]. Not a Tesla App, 2026. [cit. 2026-03-05]. Dostupné z: <https://www.notateslaapp.com/news/3527/tesla-launches-unsupervised-robotaxi-rides-in-austin>.
13. SRINIVAS, S.; RAMACHANDIRAN, S.; RAJENDRAN, S. Autonomous robot-driven deliveries: A review of recent developments and future directions. *Transportation Research Part E: Logistics and Transportation Review* [online]. 2022, **165** [cit. 2026-03-07]. ISSN 1366-5545. Dostupné z DOI: <https://doi.org/10.1016/j.tre.2022.102834>.
14. KOETSIER, J. *Starship's 2,700 Robots Have Made 9M Deliveries. Now They're Coming To Your City* [online]. Forbes, 2025. [cit. 2026-03-07]. Dostupné z: <https://www.forbes.com/sites/johnkoetsier/2025/10/15/starships-2700-robots-have-made-9m-deliveries-now-theyre-coming-to-your-city/>.
15. *Arduino® Nano ESP32* [SKU: ABX00083]. Arduino®, [b.r.]. Dostupné tiež z: <https://docs.arduino.cc/resources/datasheets/ABX00083-datasheet.pdf>.
16. *ESP32-CAM Development Board* [SKU: DFR602]. DFRobot, [b.r.]. Dostupné tiež z: https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0602_Web.pdf.
17. *OV2640 Color CMOS UXGA (2.0 MegaPixel) CAMERA CHIPTM with OmniPixel2TM Technology*. OmniVision, [b.r.]. Dostupné tiež z: https://www.uctronics.com/download/cam_module/OV2640DS.pdf.
18. *Ultrasonic Ranging Module HC - SR04*. ElecFreaks, [b.r.]. Dostupné tiež z: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.
19. *DRV8833 Dual H-Bridge Motor Driver* [SKU: SLVSAR1E]. Texas Instruments, [b.r.]. Dostupné tiež z: <https://www.ti.com/lit/ds/symlink/drv8833.pdf?ts=1773296972820>.
20. STROUSTRUP, B. *C++ Programming Language* [online]. 4. vyd. Addison-Wesley, 2013 [cit. 2026-03-16]. ISBN 978-0-321-56384-2. Dostupné z: https://chenweixiang.github.io/docs/The_C++_Programming_Language_4th_Edition_Bjarne_Stroustrup.pdf.

21. KUHLMAN, D. *A Python Book: Beginning Python, Advanced Python, and Python Exercises* [online]. Platypus Global Media, 2011 [cit. 2026-03-16]. ISBN 978-0984221233. Dostupné z: https://web.archive.org/web/20120623165941/http://cutter.rexx.com/~dkuhlman/python_book_01.html.
22. KARI PULLI Anatoly Baksheev, K. K.; ERUHIMOV, V. Realtime Computer Vision with OpenCV. *Queue* [online]. 2023, **10**(4) [cit. 2026-03-16]. ISSN 1542-7730. Dostupné z DOI: 10.1145/2181796.
23. *Open Source Computer Vision Library* [online]. OpenCV, 2025. [cit. 2026-03-16]. Dostupné z: <https://github.com/opencv/opencv>.
24. HARRIS, C. R. et al. Array programming with NumPy. *Nature*. 2020, **585**, 357–362. Dostupné z DOI: 10.1038/s41586-020-2649-2.
25. VENKATESAN Ragav; Li, B. *Convolutional Neural Networks in Visual Computing: A Concise Guide* [online]. CRC Press, 2017 [cit. 2026-04-06]. ISBN 978-1-351-65032-8. Dostupné z: <https://books.google.sk/books?id=bAM7DwAAQBAJ>.
26. CIREGAN, D.; MEIER, U.; SCHMIDHUBER, J. Multi-column deep neural networks for image classification [online]. 2012 [cit. 2026-04-06]. Dostupné z DOI: 10.1109/CVPR.2012.6248110.
27. TERVEN, J.; CÓRDOVA-ESPARZA, D.-M.; ROMERO-GONZÁLEZ, J.-A. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction* [online]. 2023, **5**(4), 1680–1716 [cit. 2026-03-16]. ISSN 2504-4990. Dostupné z DOI: 10.3390/make5040083.
28. JOCHER, G.; QIU, J.; CHAURASIA, A. *Ultralytics YOLO* [online]. 2023. [cit. 2026-03-16]. Dostupné z: <https://github.com/ultralytics/ultralytics>.

Použitie nástrojov umelej inteligencie

Google (2026), Gemini 3 Fast, celý text, úprava štylizácie a reformulácia textu.

Google (2026), Gemini 3.1 Flash, abstrakt, preklad textu do angličtiny.

OpenAI (2026), GPT-5.4, zdrojový kód, generovanie základnej štruktúry kódu pre Arduino, ESP32-CAM a server v jazykoch C++ a Python.

Dodatok A: Výpis dlhého kódu

V prílohe sa nachádza archív zdrojovy_kod.zip so zdrojovými kódmi pre vozidla a server. Zdrojový kód projektu je dostupný aj na:

<https://github.com/ruddpj/BP-AutonomousVehicle.git>.

Arduino programy

- Arduino/
 - ArduinoController/ArduinoController.ino
 - WifiCamera/WifiCamera.ino

Python programy

- Python/
 - connect.py
 - detect.py
 - interpret.py
 - main.py
 - remote.py
 - transform.py

Dodatok B: Výsledky testov

V prílohe sa nachádza archív `vysledky.zip`, ktorý obsahuje výsledné dáta z testovania vo formáte `.ods` (alternatívne `.xls`). Súčasťou archívu sú aj dizajny prekážok a trate, ktoré boli pri testovaní použité.